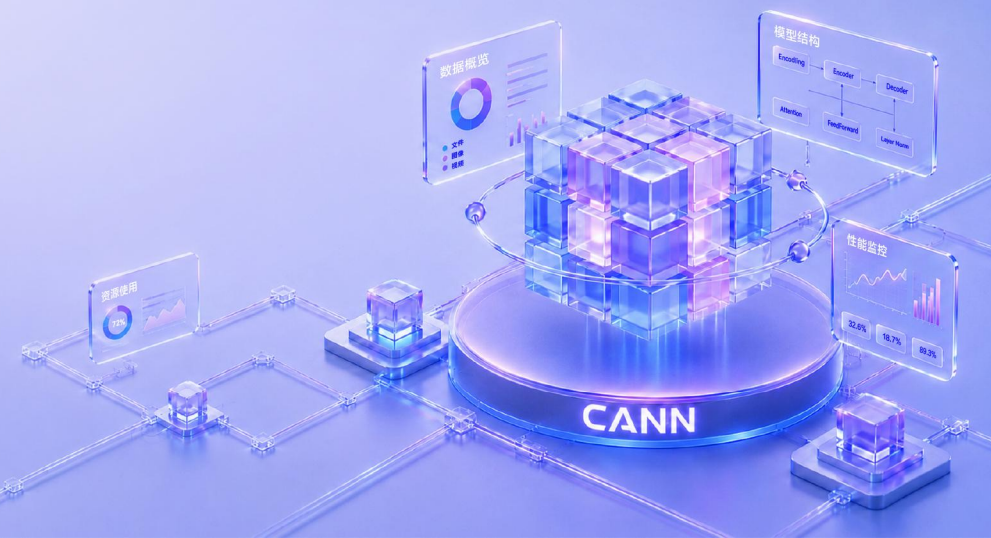


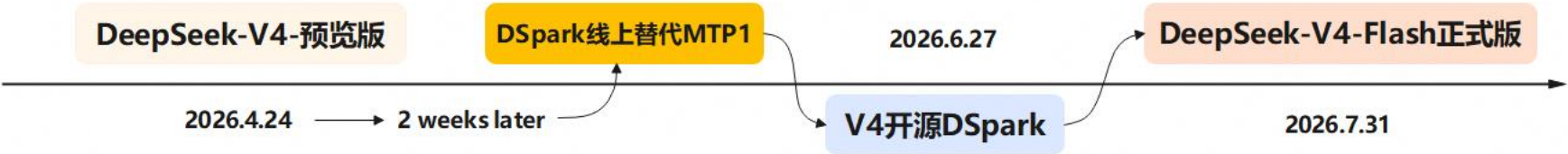


DSV4-Flash 昇腾950DT推理： DSpark特性实现与性能优化

王正平



DeepSeek-V4 采用 MoE 架构，并引入结合 CSA 与 HCA 的混合注意力机制和 mHC，原生支持百万 Token 上下文，在降低长上下文计算与 KV Cache 开销的同时，进一步提升知识、推理、代码及 Agent 能力



DeepSeek-V4 原有的 MTP-1 投机长度较短、加速空间有限，而增加投机长度又会带来接受率下降和验证开销。为此，DeepSeek 引入 **DSpark**，通过提升草稿质量并动态调整验证长度，在兼顾系统吞吐的同时提高生成速度。

deepseek-ai/DeepSeek-V4-Flash-DSpark
Text Generation • 165B • Updated Jul 4 • 519k • 243

→ DeepSeek-V4-Flash预览版

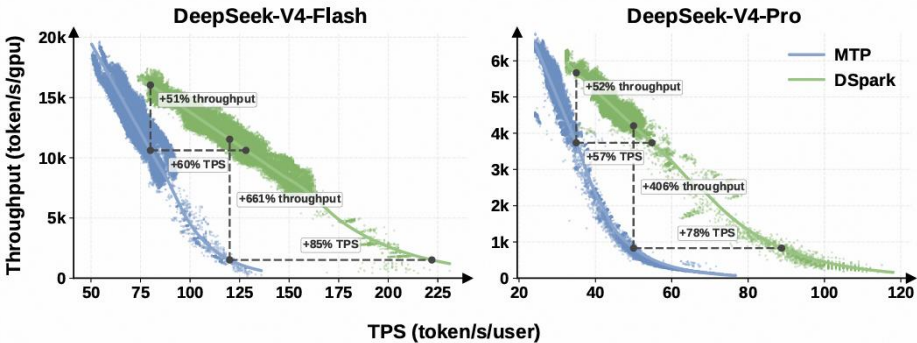
deepseek-ai/DeepSeek-V4-Pro-DSpark
Text Generation • 1.7T • Updated Jul 4 • 47.3k • 535

→ DeepSeek-V4-Pro预览版

deepseek-ai/DeepSeek-V4-Flash-0731
Text Generation • 304B • Updated 4 days ago • 433k • 2.3k

→ DeepSeek-V4-Flash正式版
(后训练，权重更新)

DSV4开源权重 [1]



DSpark 官方性能提升 [2]
Flash 60%-85% ↑ Pro 57%-78%
↑

[1] <https://huggingface.co/deepseek-ai/DeepSeek-V4-Flash-0731>
[2] Cheng, X., Yu, X., Shao, C., et al. DSpark: Confidence-Scheduled Speculative Decoding with Semi-Autoregressive Generation. arXiv preprint arXiv:2607.05147, 2026.



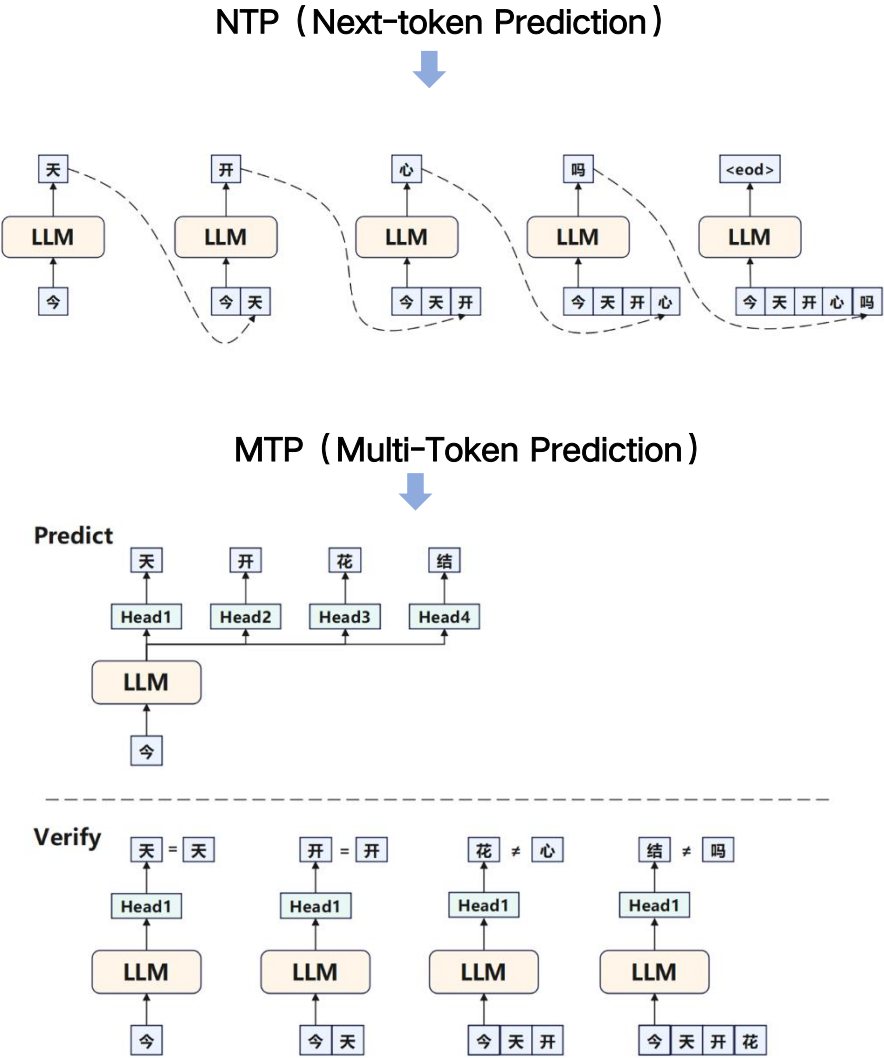
目录

Part 1 投机推理与DSpark原理介绍

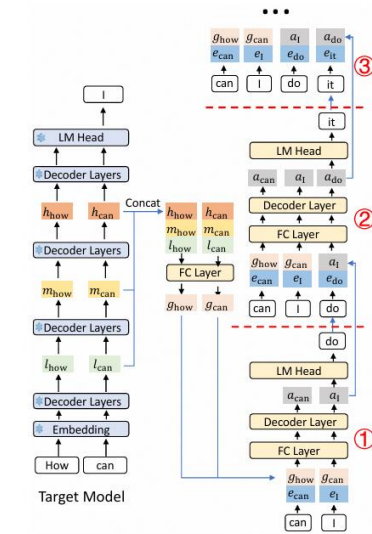
Part 2 recipes infer的DSpark实现与性能优化

Part 3 Future Plan

投机推理是一种通过一次性预测并验证多个Token加速Decode的方法



投机推理中的 Draft 模型已经从逐 Token串行的预测方式，演进到更高并行粒度的块级生成。MTP 将多 Token 预测能力内生到主模型；EAGLE3 进一步利用 Target 多层特征融合与直接 Token 预测；DFlash 则引入 Block Diffusion，在一次 Draft Forward 中并行生成完整候选块。



EAGLE-3[2]

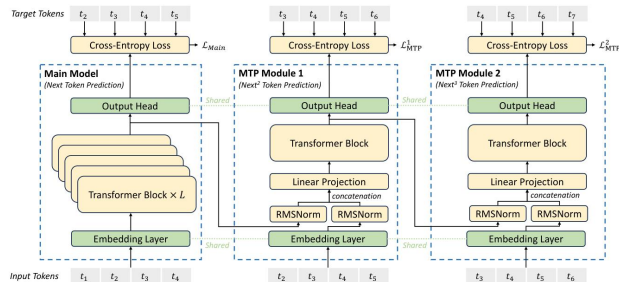
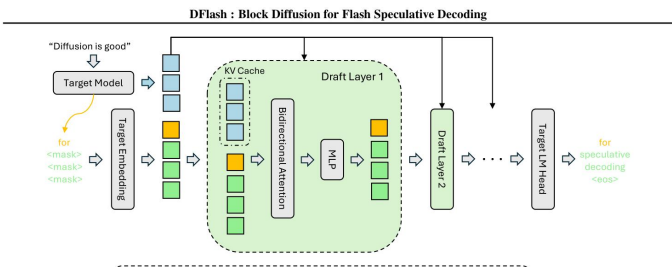


Figure 3 | Illustration of our Multi-Token Prediction (MTP) implementation. We keep the

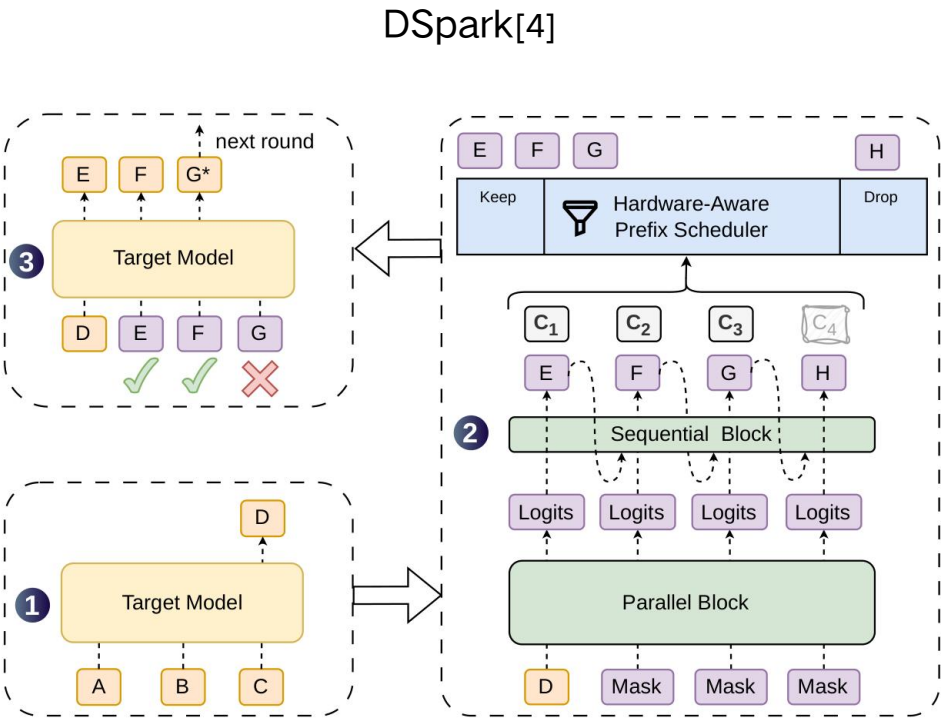
• deepseek-v3-mtp[1]



• DFlash[3]

[1] DeepSeek-AI, “DeepSeek-V3 Technical Report,” arXiv:2412.19437, 2024.
[2] Li et al., “EAGLE-3: Scaling up Inference Acceleration of Large Language Models via Training-Time Test,” arXiv:2503.01840, 2025.
[3] Chen et al., “DFlash: Block Diffusion for Flash Speculative Decoding,” ICML 2026.

DSpark把主要计算并行完成，再用轻量顺序模块补回 Token 之间的依赖关系



一轮DSpark的推理流程：

- 1. 并行预测：一次计算多个候选Token的基础分布；
- 2. 顺序修正：利用已经生成的前缀修正后续Token；
- 3. 置信度筛选：只将有价值的连续前缀送给Target验证；
- 4. 批量验证：Target一次验证多个Token并提交接受结果。

Figure 1 | The DSpark architecture and decoding cycle. Given prompt tokens **ABC**, the target model executes one step to generate the next token **D**, which serves as the anchor for

★ DSpark 将传统固定长度的单 Token Decode，转变为包含**动态草稿**、**变长验证**和**状态回退**的推理流程，对 CANN的动态图执行、Attention 算子、KV Cache 管理以及计算通信协同能力提出了新的挑战。

[4]Cheng, X., Yu, X., Shao, C., et al. DSpark: Confidence-Scheduled Speculative Decoding with Semi-Autoregressive Generation. arXiv preprint arXiv:2607.05147, 2026.



目录

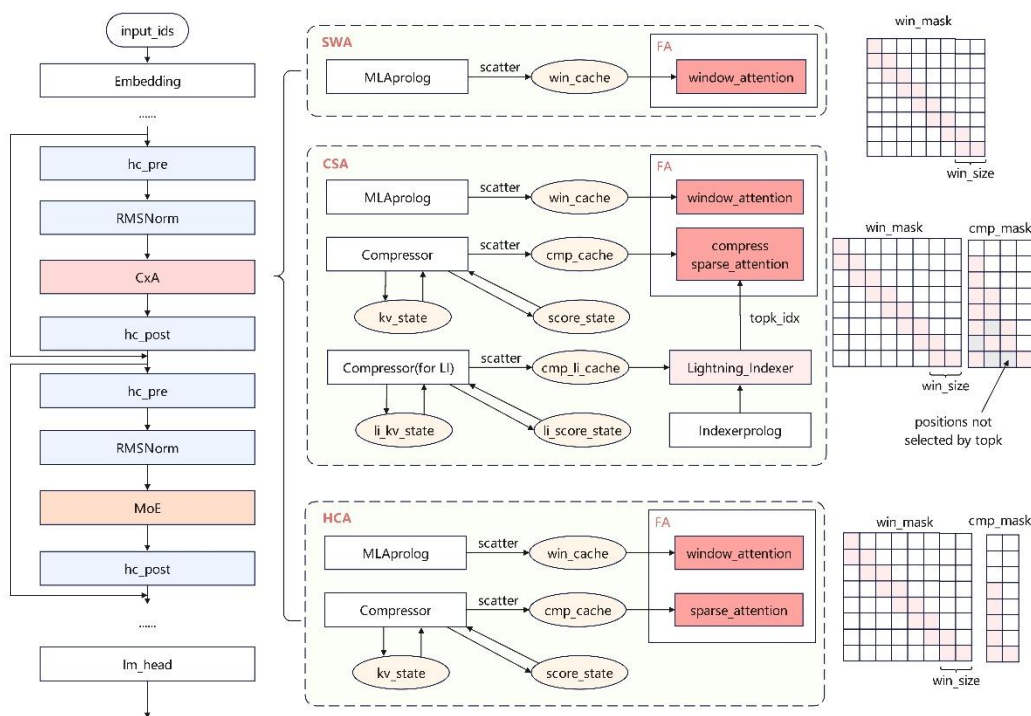
Part 1 投机推理与DSpark原理介绍

Part 2 recipes infer的DSpark实现与性能优化

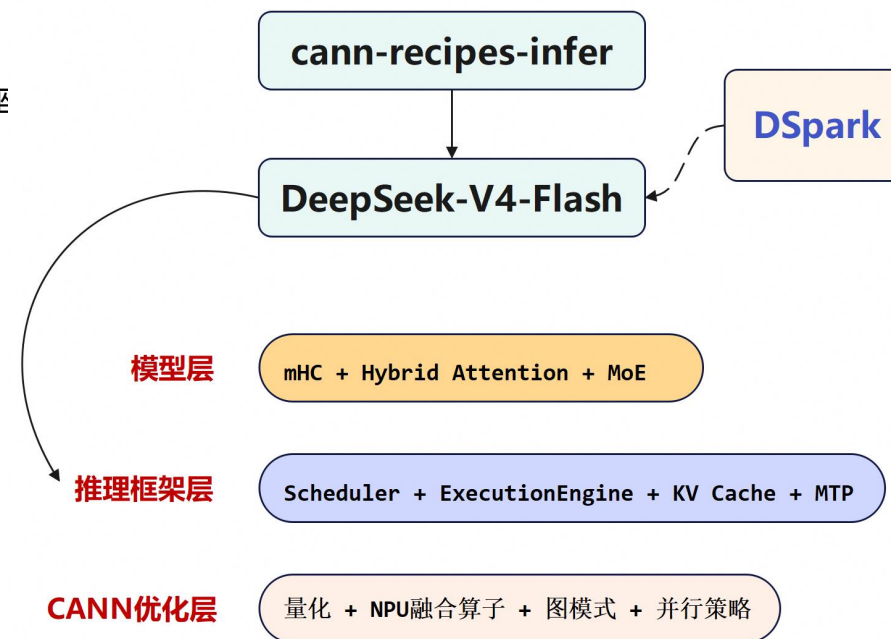
Part 3 Future Plan

在cann-recipes-infer的DSV4-Flash网络中接入DSpark方案

recipes-infer 0 Day支持了DeepSeek-V4的模型推理部署，目前适配支持Atlas-A3 Pod和950PR/DT等多代际昇腾芯片，提供长达1M序列的高性能推理能力，是已有的DSV4优化底座



• Infer仓上DeepSeek-V4模型的结构



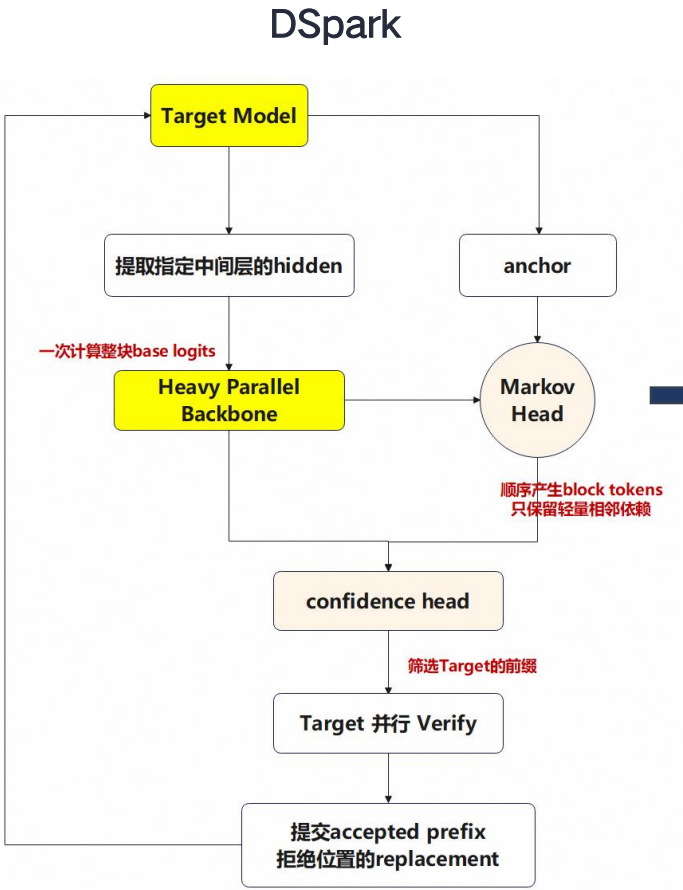
recipes-infer 的已有状态:

- 支持DeepSeek-V4 Flash在昇腾950PR/DT平台运行;
- 已有统一的请求调度、批处理和KV Cache管理;
- 已支持MTP投机推理;
- 已具备NpuGraphEx、量化、融合算子和并行执行基础;

Dspark的Proposal生成方式以及运行时的契约相比MTP有一些变化



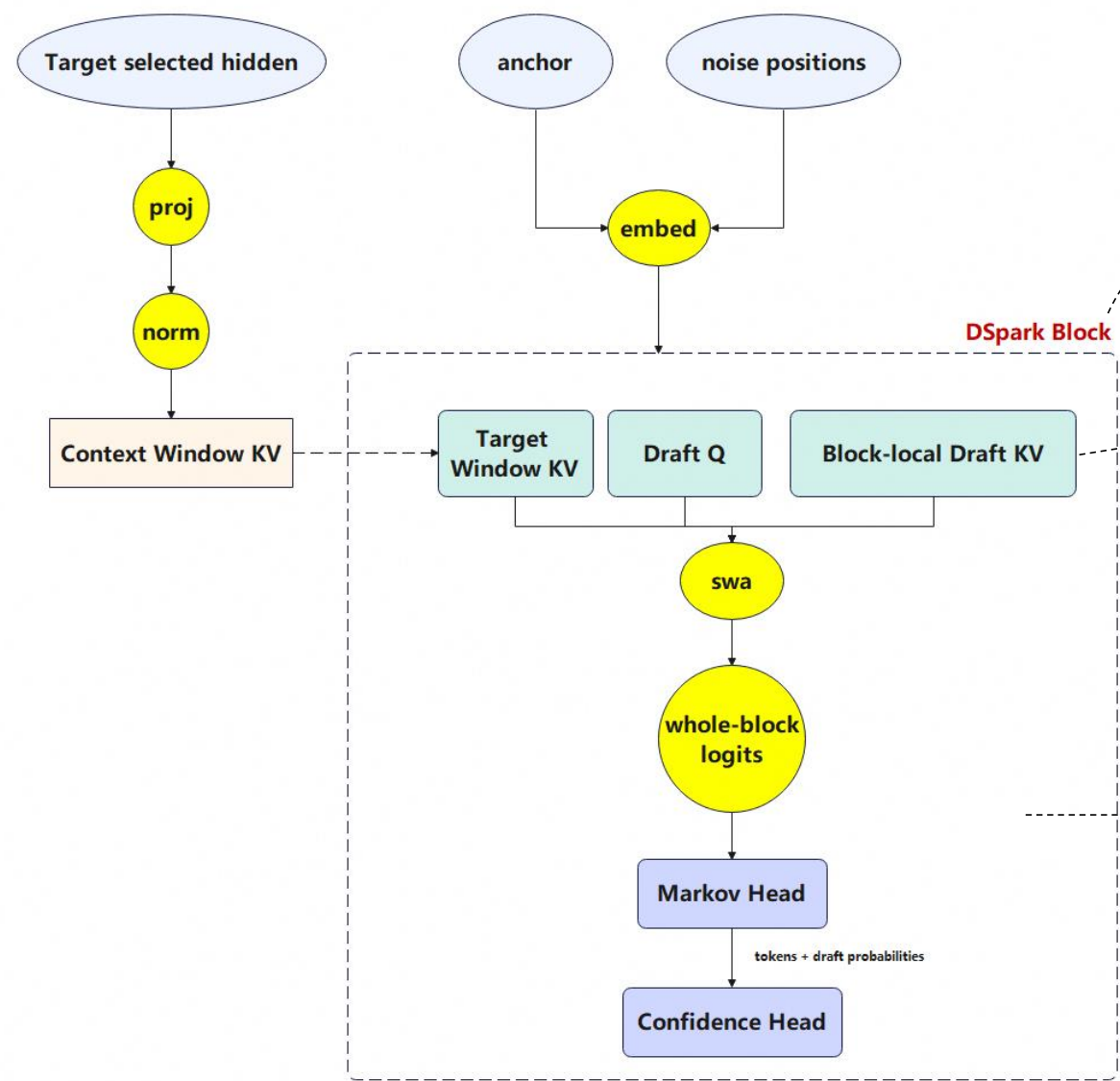
MTP的主循环是多次轻量模型
Forward得到固定的候选



Dspark的主循环是一次块级别的Backbone、
一些顺序Head和一个变长的Verify

这些差异带来了新的诉求：

差异	诉求	解决方案
独立块级 Proposal	独立的Proposal调度	DSparkWorker
Target 多层特征依赖	Target-Draft 交互	Hidden States 传递
动态 Verify 长度	变长图调度	proposal_lens + q_len 分档
概率接受机制	自定义验证策略	Rejection Sampling
独立跨轮状态	请求级状态管理	DraftState
独立 Window KV	双 Cache 管理	Paged KV + Ring KV

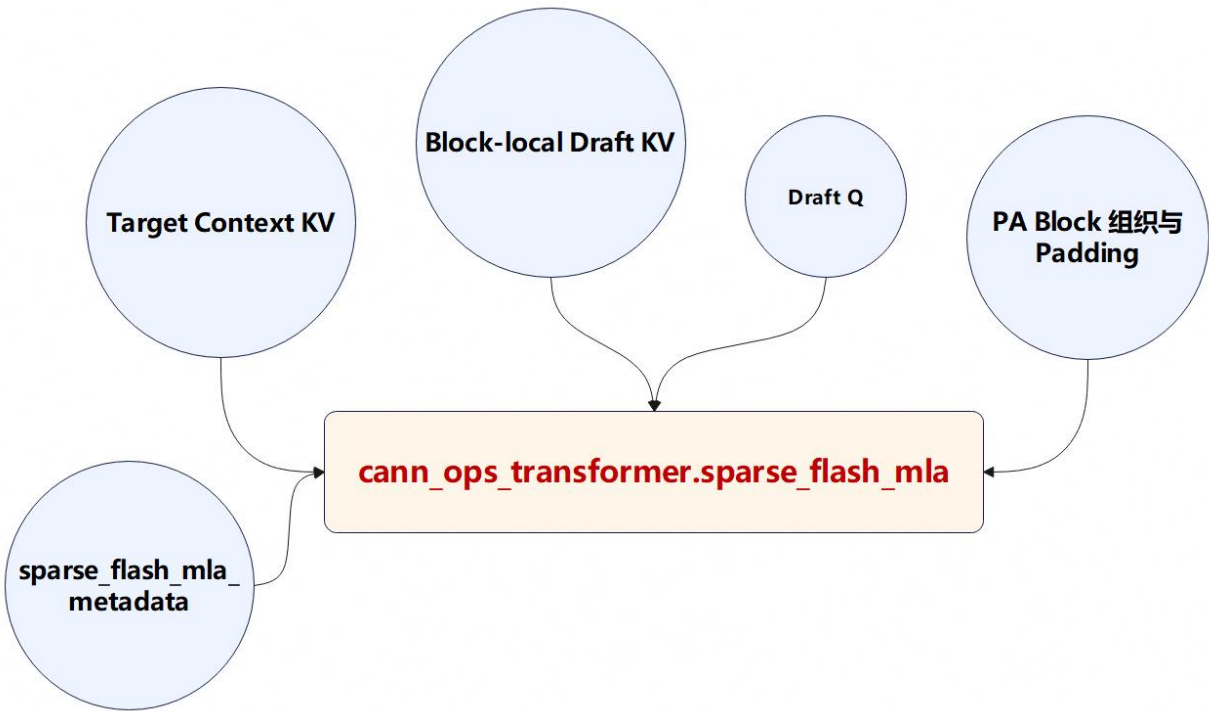


• Proposal 逻辑结构示意图

框架改造解决“如何接入”，Proposal 优化进一步解决“如何高效执行”

- 多层 DSpark Backbone可以跨层复用一些公共计算
- Attention的Q、KV分支是独立的，存在多流并行的可能
- Attention肯定需要一个融合算子来加速
- FA算子的Metadata准备可以与不冲突的计算做并行
- Proposal需要入图才能获得最佳性能
- ...

SparseFlashMLA: 将 Window KV 与 Block-local KV 映射为统一稀疏 Attention



针对DSpark场景的适配

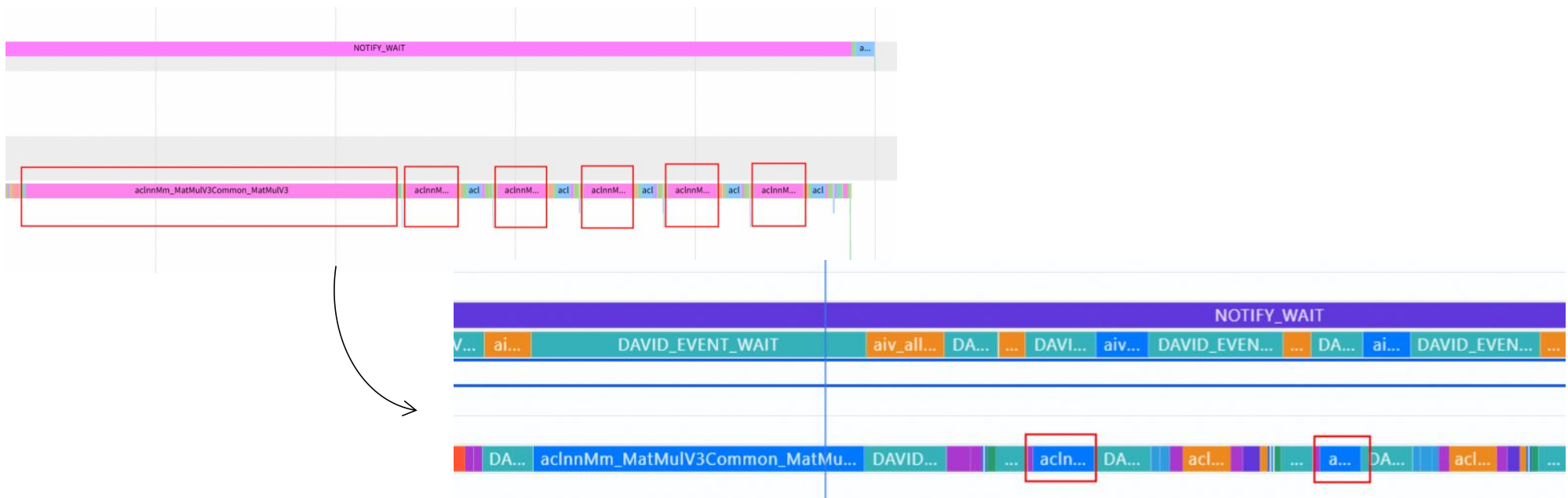
环节	适配方式
历史上下文	将 Target Ring KV 作为 Window Context
当前候选块	将 Block-local Draft KV 追加到 Context KV
KV 存储	转换为 PA_BBND 布局并生成 Block Table
有效长度	物理空间按 Block 补齐，逻辑长度由 sequested_ori_kv 保留
滑动窗口	配置 ori_win_left=127，对应 Window Size=128
执行模式	支持 NpuGraphEx，并与 Metadata/Q-KV 多流配合

SparseFlashMLA 本身并非 DSpark 专用算子，接口支持不同 KV 来源、Block Table、稀疏模式和有效序列长度。DSpark 通过参数配置，可以将其适配为 “Target Window KV + Draft Block KV” 的 Attention。



通过对Proposal图模式下的推理优化，一次DSpark的Forward开销从5.3ms减小到1.7ms

③ lm_head的TP并行策略——0.2ms ↑



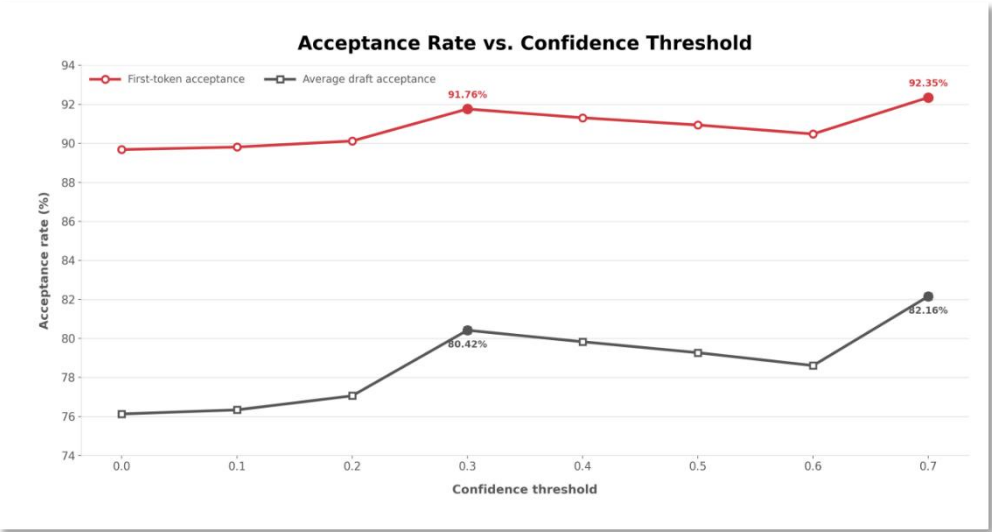
② 多流并行：每层 Attention 内，Q 分支与 KV 分支并行，直到 FA 前再汇合——0.8ms ↑



DSV4-Flash-Dspark初步实现89%的首token接受率和76%的平均接受率

在统一测试环境、模型配置及统计口径下：

基底	硬件环境	并行策略	数据集	方案	等效TPOT	生成效率
DSV4-Flash	950DT	4卡EP并行	MATH-500	MTP1	9.83ms	1.00x
				DSpark	3.93ms	2.50x ↑



- Confidence_threshold 测试实验

组合收益估算：

- Confidence_threshold的开启：8% ↑
- Cann-Super Kernel特性的使能：12% ↑
- TPOT: 3.93ms → 3.24ms

目录

Part 1 投机推理与DSpark原理介绍

Part 2 recipes infer的DSpark实现与性能优化

Part 3 Future Plan

- Confidence 拦截功能的开源（8月中旬）；
- DeepSeekV4-Flash正式版的DSpark开源（8月下旬）；
- Atlas-A3 Pod 系列芯片的DSpark开源（9月上旬）；



扫码参与社区互动答疑贴



微信群小助手

Thank you.

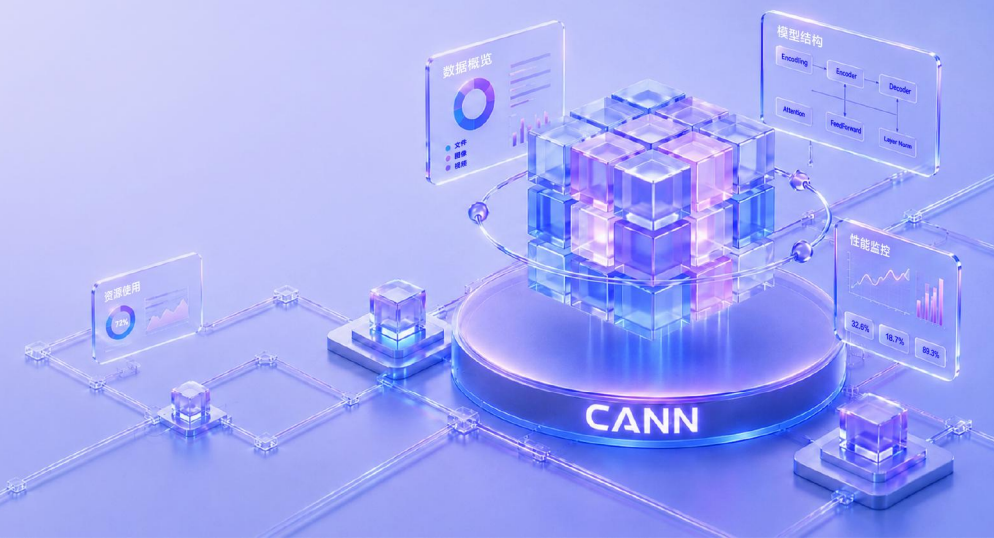
社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.

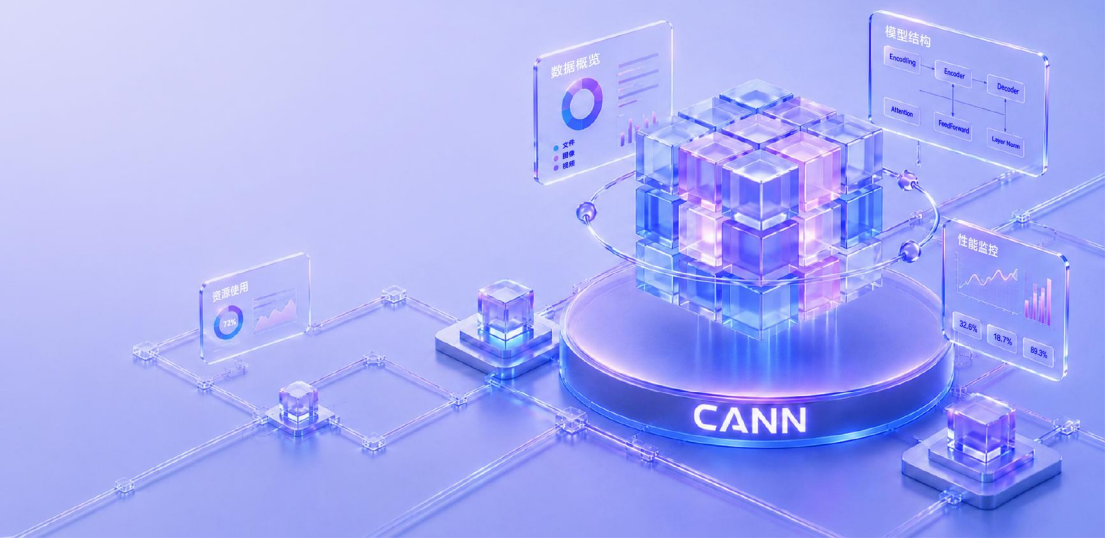
<https://gitcode.com/cann>





CANN支持Longcat-2.0长序列高效推理

葛根华



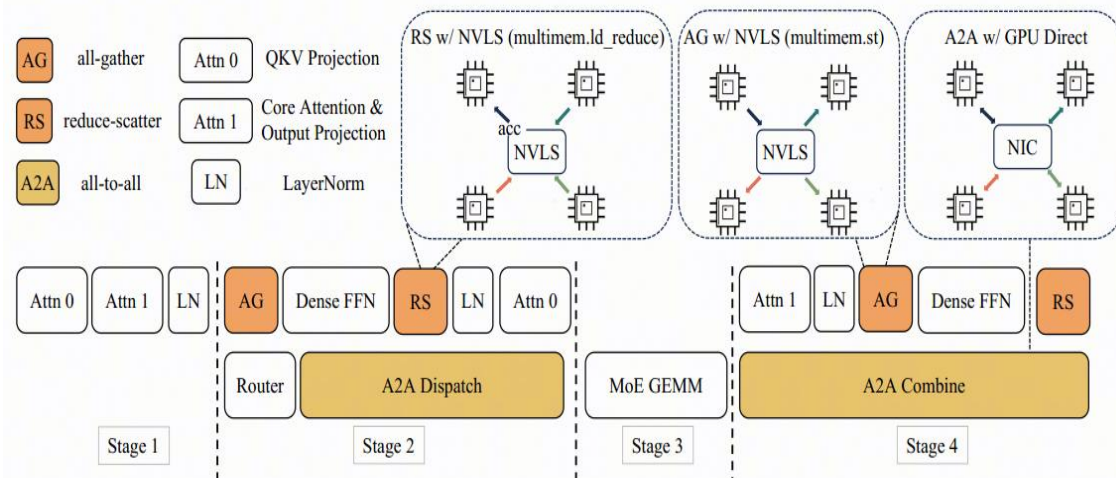
目录

Part 1 模型结构、部署策略

Part 2 推理优化技术

Part 3 Future Plan

模型结构(一) - Overview



➤ ShortCut-MOE结构

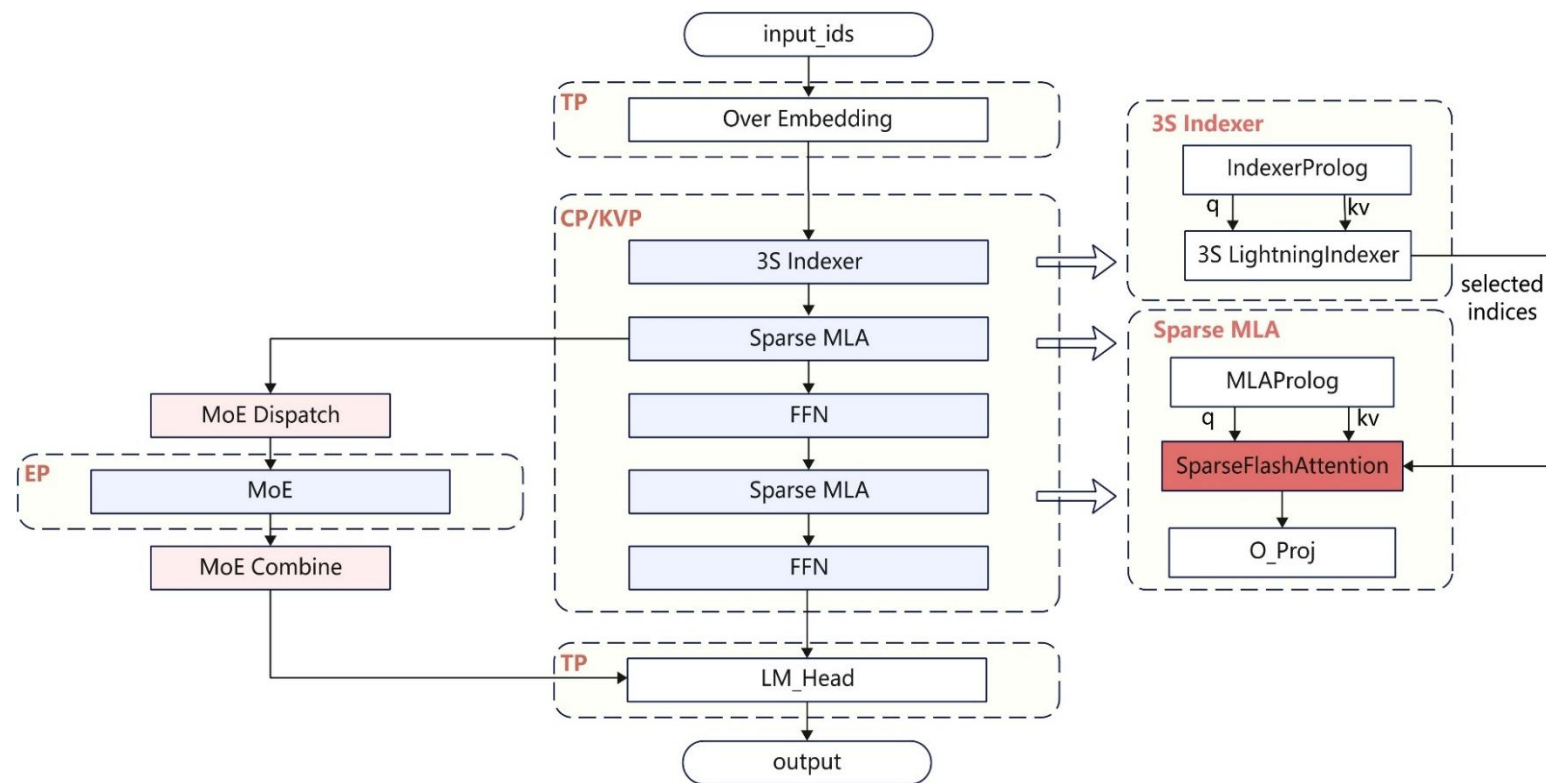
- 每个Layer包含两轮Attention与FFN计算，和一轮MOE计算
- 保留零计算专家结构，在重要性较低的Token上节省算力

➤ 相较于此前发布的LongCat-Flash, LongCat-2.0 模型全方位增大，总参数量达到1.6T

- 层数：28->38
- 路由专家数量：512->768
- 隐藏层维度：6144->8192

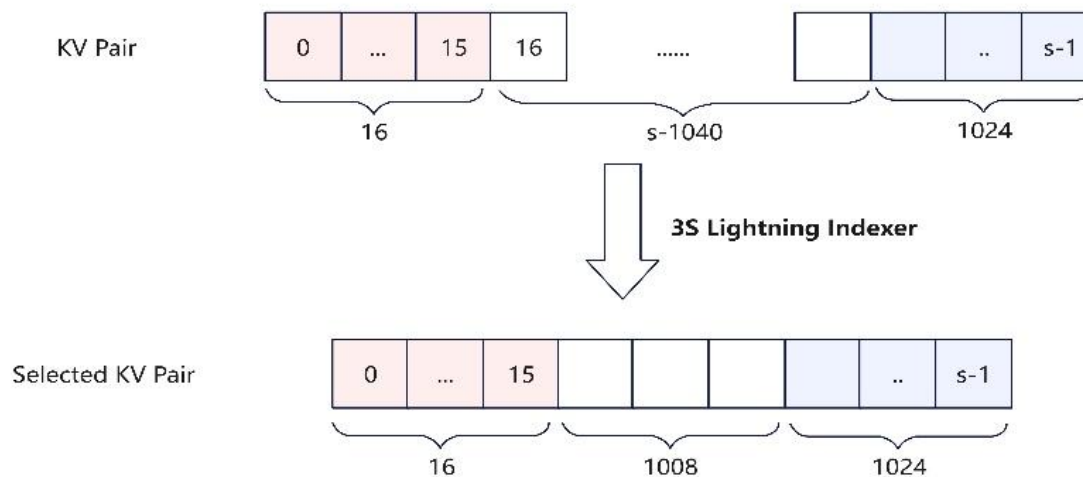
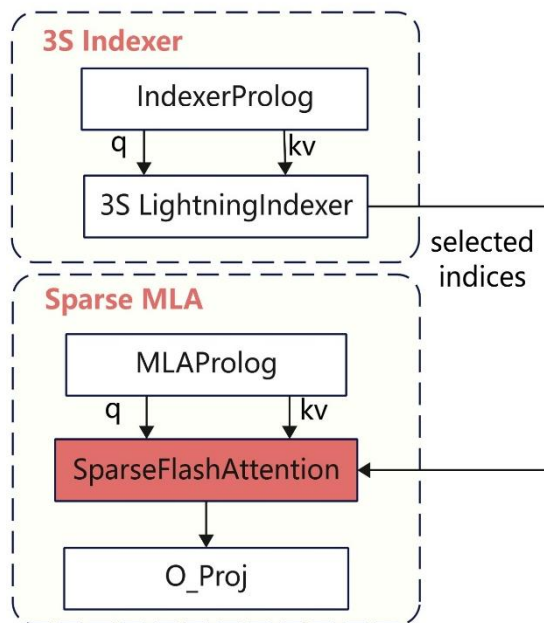
➤ LongCat-2.0在MOE/MLP部分原生支持Int8量化的前提下，模型的显存占用达到了2.1TB，模型能力增强的同时，为推理部署与性能优化带来新的挑战

- A2 EP128部署，TPOT时延可达20ms，支持1M超长序列

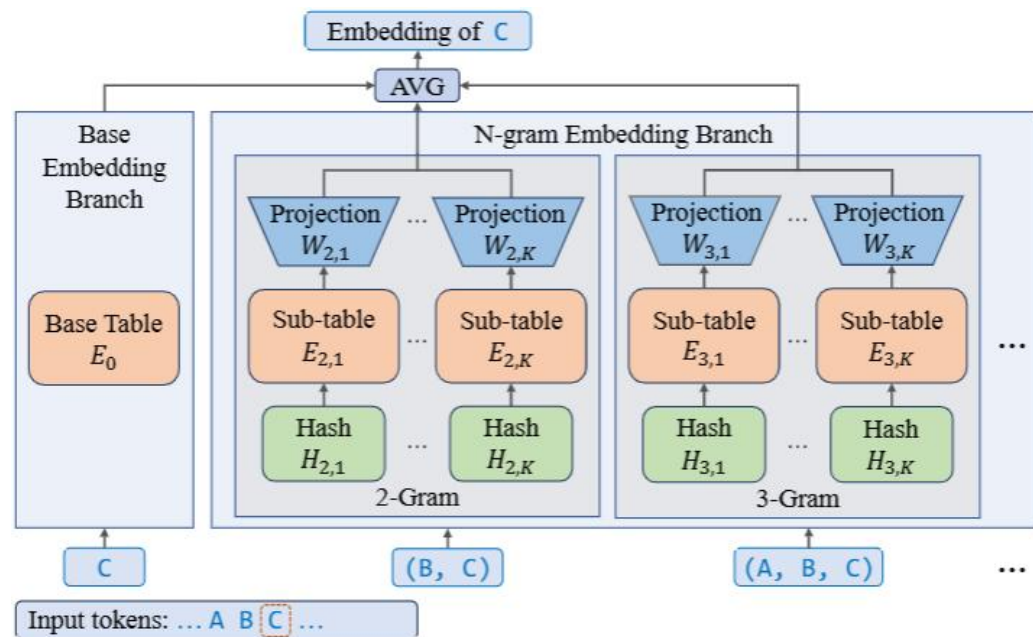


模型结构(二) - Attention稀疏与滑窗

- DSA稀疏结构
- 在Attention计算之前，使用Lightning Indexer为每个token选择 Top_k KV
 - 在ScMOE结构下，每层的两个SFA共用一组Topk_indices
- Sink Sliding Window(3S)
 - 固定选择每段请求的init 16和local 1024，再从中段选择Topk 1008组成TopK=2048个KV



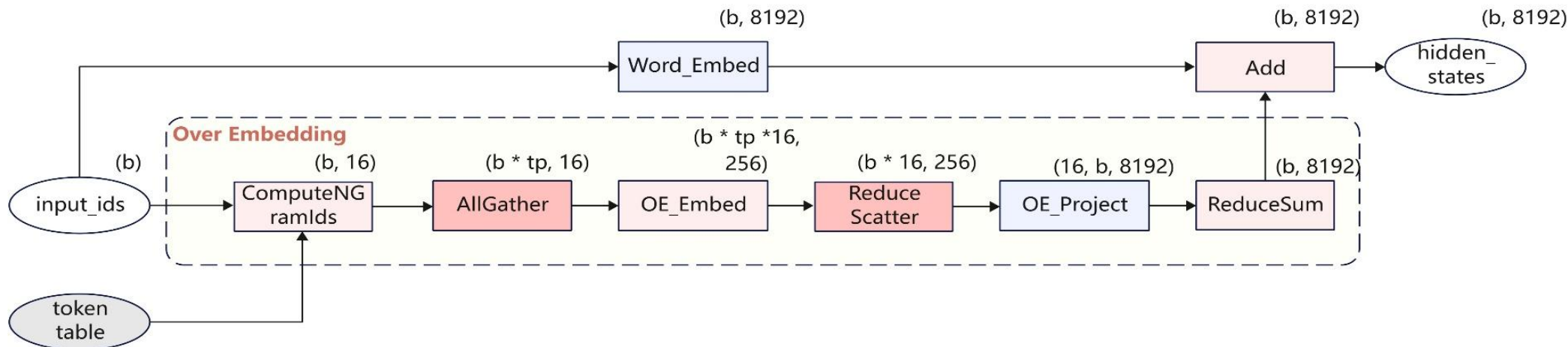
模型结构(三) - Over Embedding即N-Gram



➤ 增加 N-Gram 统计特征表，提升模型效果

1. 为 oe_k 张子表，将每个token与前序 $oe_n - 1$ 个token通过哈希计算，得到 $(oe_n - 1) * oe_k$ 个 oe_token
2. 对 oe_token 做OE Embedding和OE Projection计算
3. 将 oe_hidden_states 与原始Embedding计算的 $hidden_states$ 加权平均

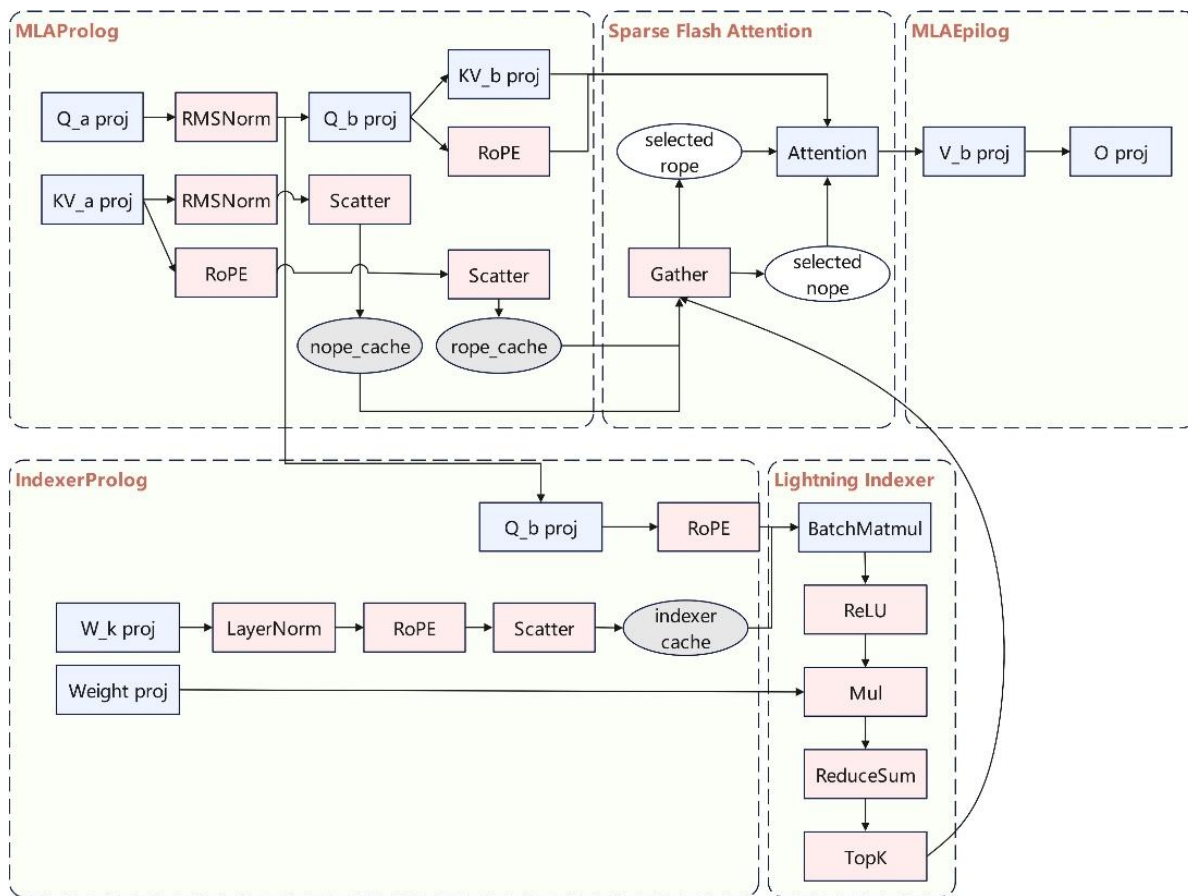
➤ 示例: $oe_n=5, oe_k=4, n_gram=(oe_n - 1) * oe_k=16$



融合算子

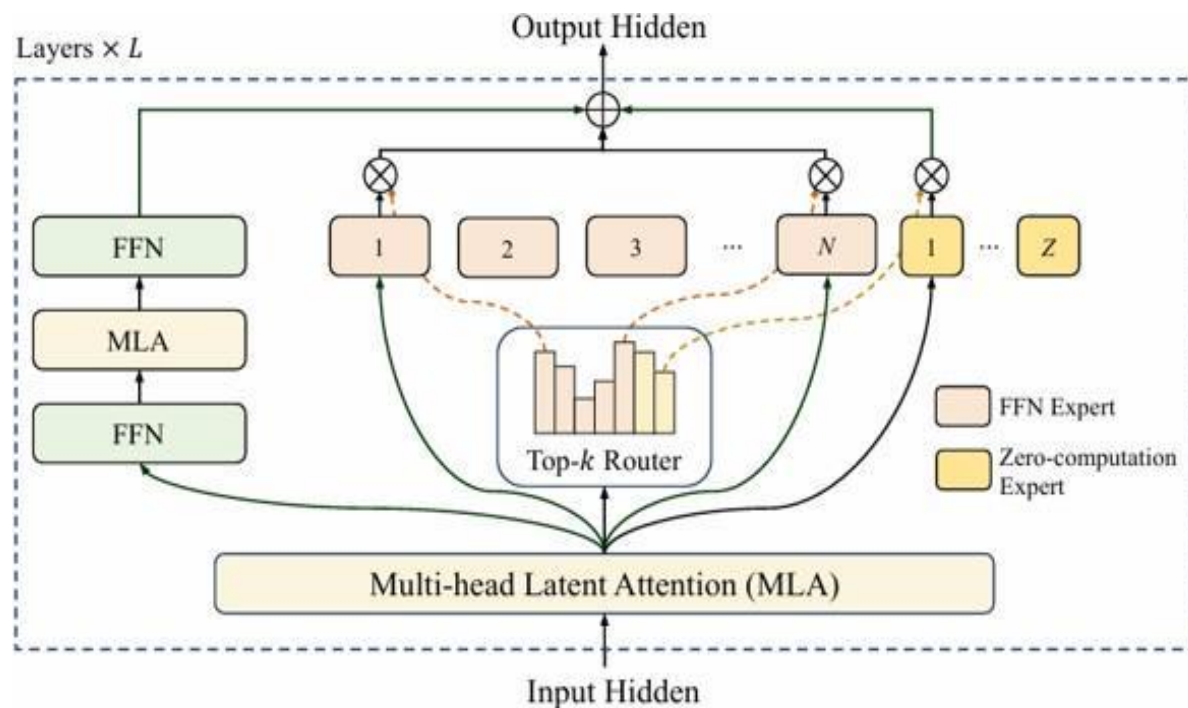
➤ Attention

- 本次开源的样例里，MLAProlog、Lightning Indexer和Sparse Flash Attention使用了NPU融合算子实现



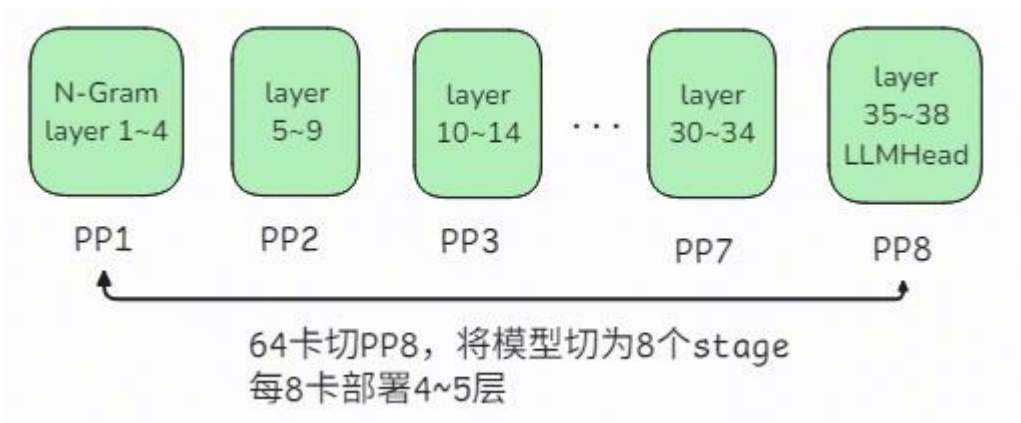
➤ MOE

- 针对LongCat系列网络MOE结构的零专家结构，Decode使用的moe_distribute和moe_combine算子支持了零专家功能



Prefill: PP+CP切分

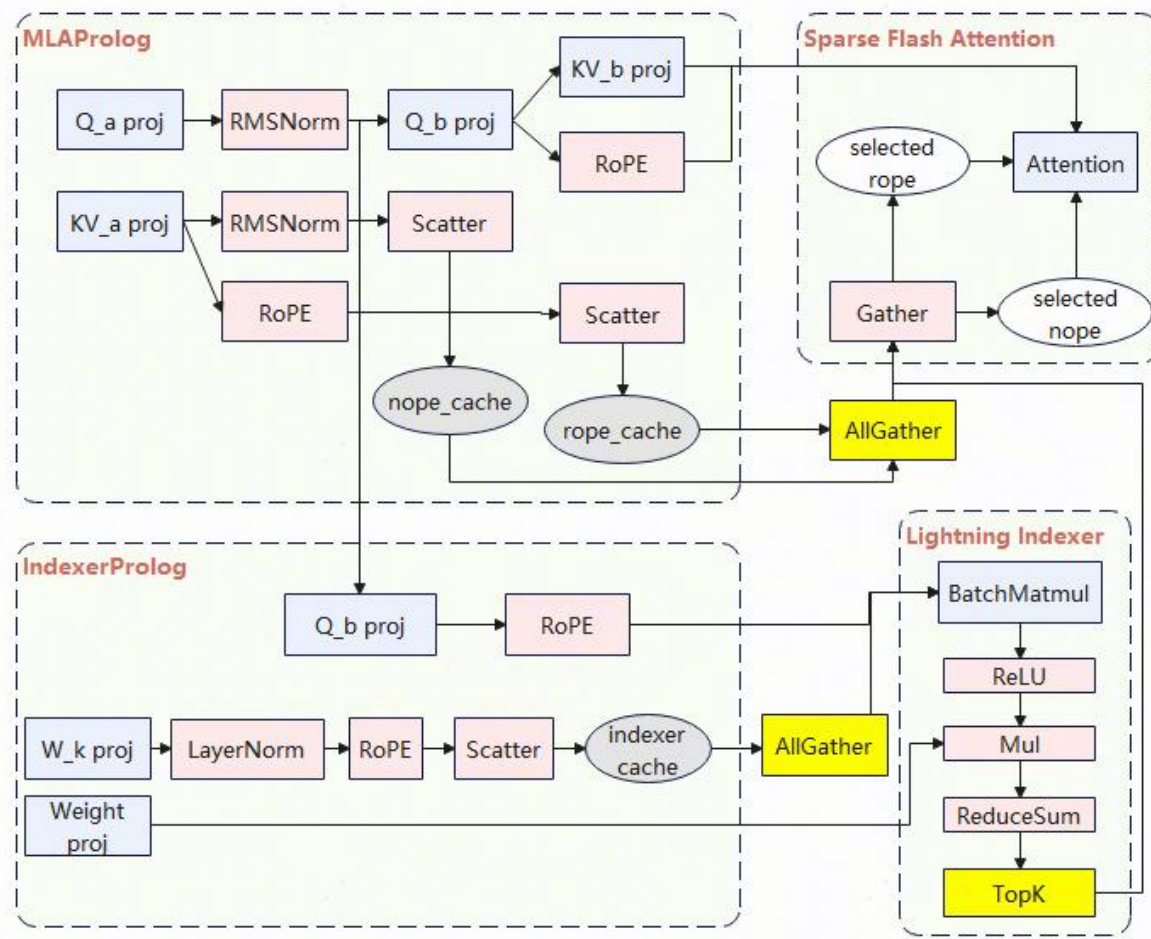
- ❑ Prefill A2 64卡部署，A2跨Server通信带宽存在瓶颈，故采用PP切分将通信域限定在Server内
 - 压缩MoE通信耗时10x倍提升整体吞吐



- ❑ 本次开源的实践做了Context Parallel Pipeline部署，将64卡Atlas A2分隔成M x N组卡，
 - 在M=8维度做Pipeline并行
 - 在N=8维度做Attention域的Context Parallel (CP)并行和MOE域的EP并行

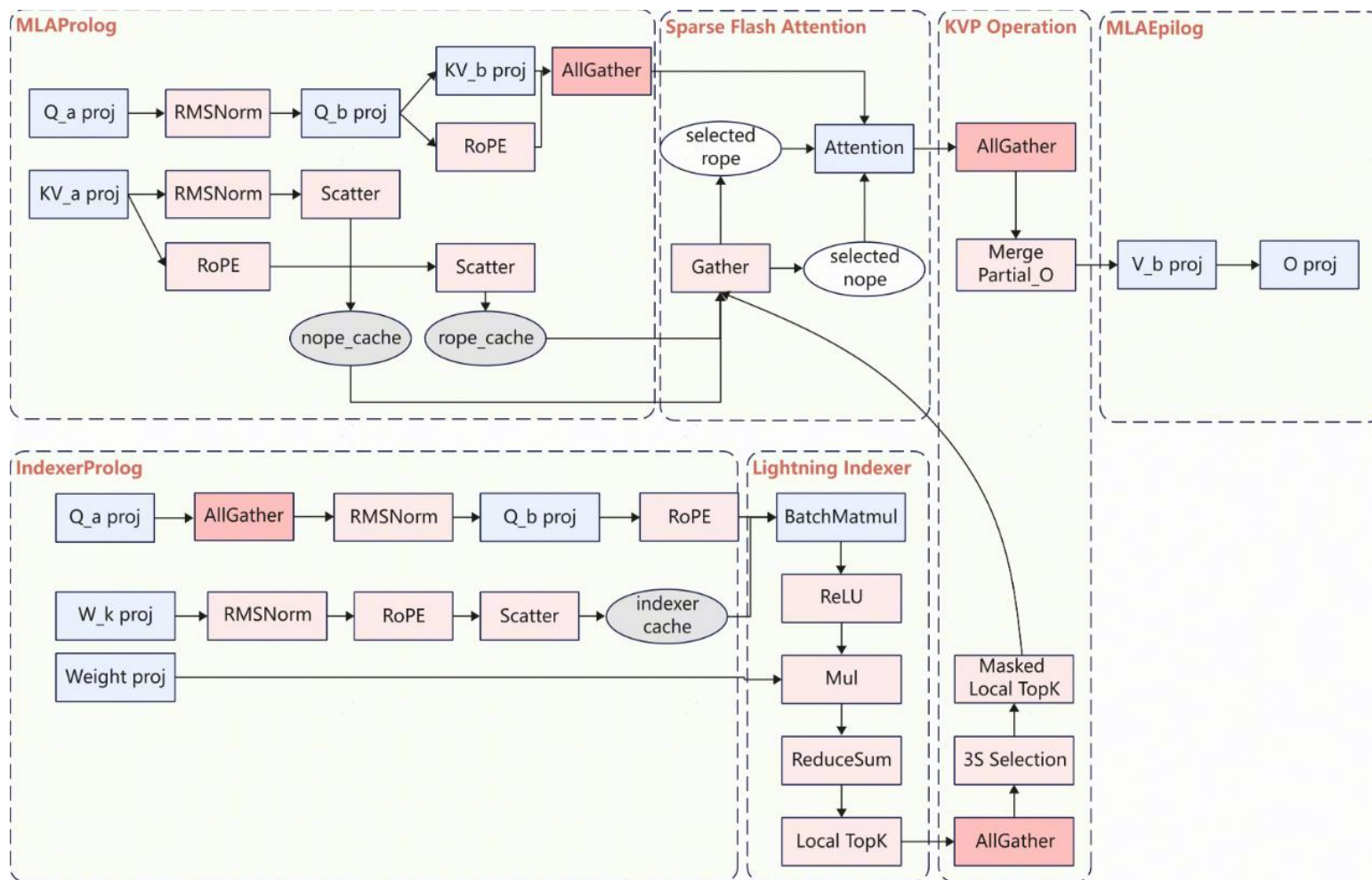
<https://gitcode.com/cann>

- ❑ DSA新增的Indexer模块(TopK)是长序列场景主要瓶颈，因此切Q使得同一时刻所有卡均摊Indexer计算，缓解Vector bound
 - 对输入请求按照序列维度并行切分
 - 在Indexer和SFA计算前，分别对Indexer Key Cache和KV Cache做AG通信
 - 每个rank用部分q和完整Cache进行LI和SFA计算

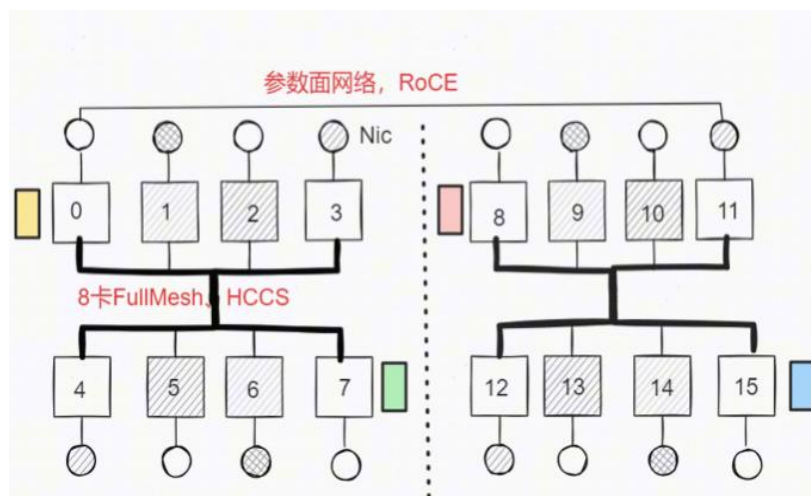


Decode: DP128+EP128 或 DP16CP8+EP128

- 针对 Decode 在请求调度层做差异化处理：短/中等长度请求采用 DP128+EP128 并行部署方案；针对超长序列，采用 DP16CP8+EP128的并行部署方案，支撑单条1M 超长请求。
- DP实际业务场景里不同请求的序列长度不一样，需要再对KV做并行切分，才能有效针对LI计算实现Rank间的负载均衡



- 对传统的全EP域AllToAll方案提出针对性优化，**将节点间低带宽通信的范围限制在原始Token**
 - 在不同节点间，local_rank_id相同的卡间做all_gather汇集Token，使得每个节点内都有全部的Token
 - 对汇集后的Token做topk_gating后，再在各自节点内做EP域的AllToAll通信和专家计算
 - 在不同节点间，local_rank_id相同的卡间做reduce_scatter，汇聚不同节点的专家信息，返还各自Token



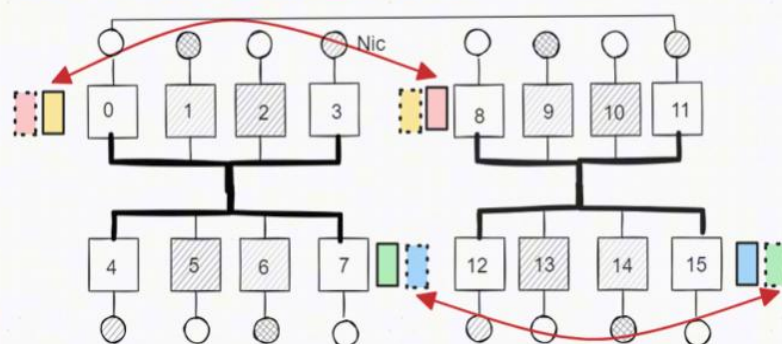
以两台Server16卡、Top K=2示意:

1、16个专家，专家号等于卡号；即0号专家E#0放0卡

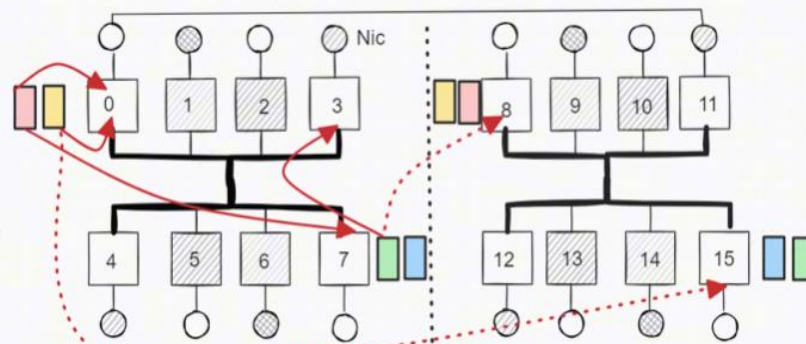
2、Token选择专家，假设:

- 代表卡0上的Token，它选择卡0上E#0 和 卡15上E#15
- 代表卡8上的Token，它选择卡0上的E#0 和 卡7上E#7
- 代表卡7上的Token，它选择卡3上的E#3 和 卡8上E#8
- 代表卡15上的Token，它选择卡8上的E#8 和 卡15上E#15

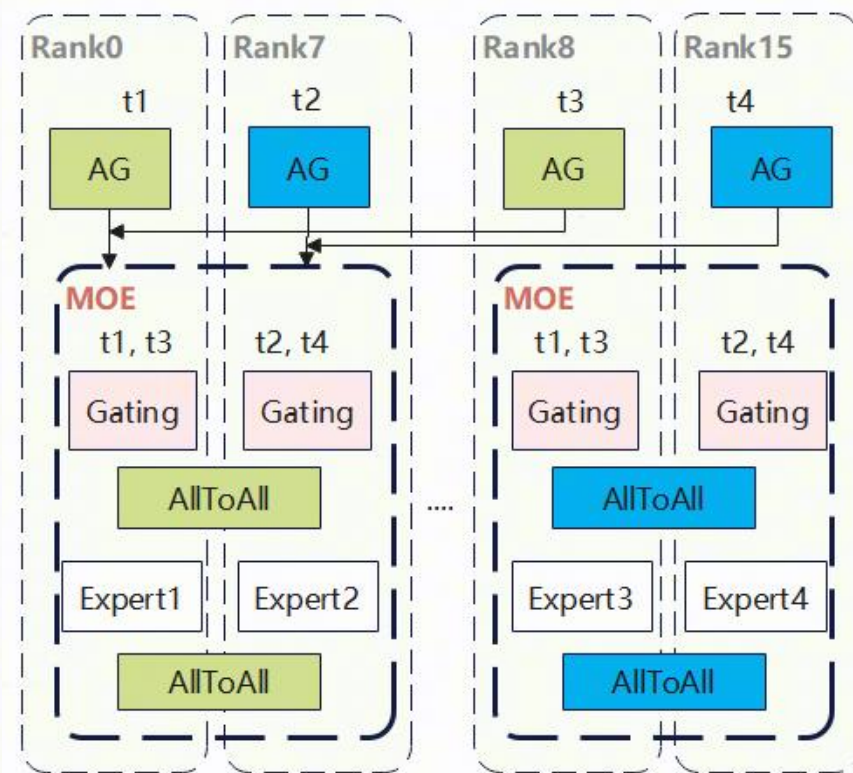
STEP1: Server间慢速互联RoCE上, 执行AllGather



STEP2: 每台Server内高速互联HCCS上, 执行All2Allv



EP域内该专家不存在, MoenitRouting支持忽略



目录

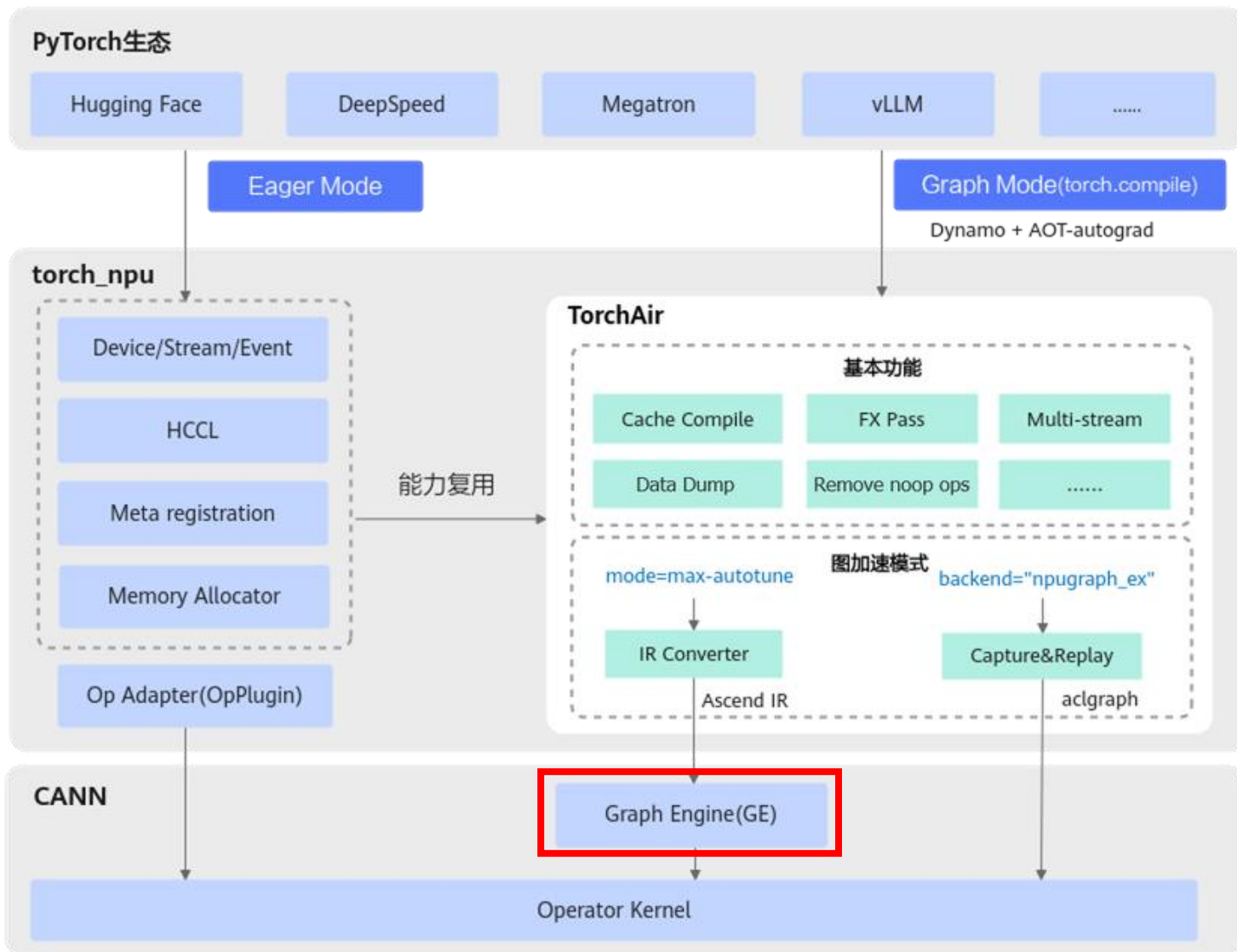
Part 1 模型切分部署策略

Part 2 推理优化技术

Part 3 Future Plan

图模式

- Decode 主模型和MTP模型静态shape图全下沉，并使用多项手段来降低TPOT



编译缓存

多bs图启动从xxMin降低到xxMax

多流并行

不同引擎的算子并行执行

控核

多流并行场景，每条流单独控核

预取

提前预取算子输入到cache

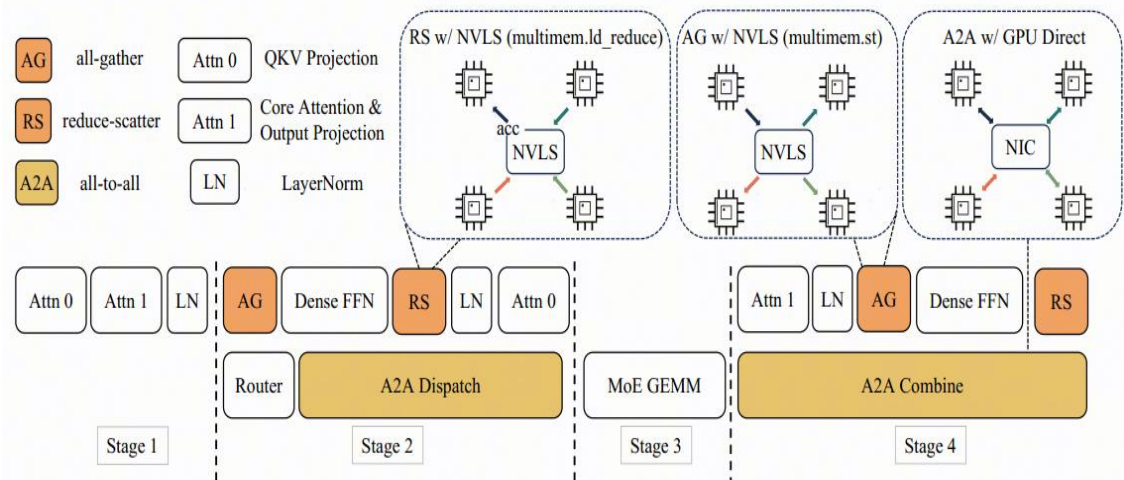
super kernel

减少多算子启动和调度开销

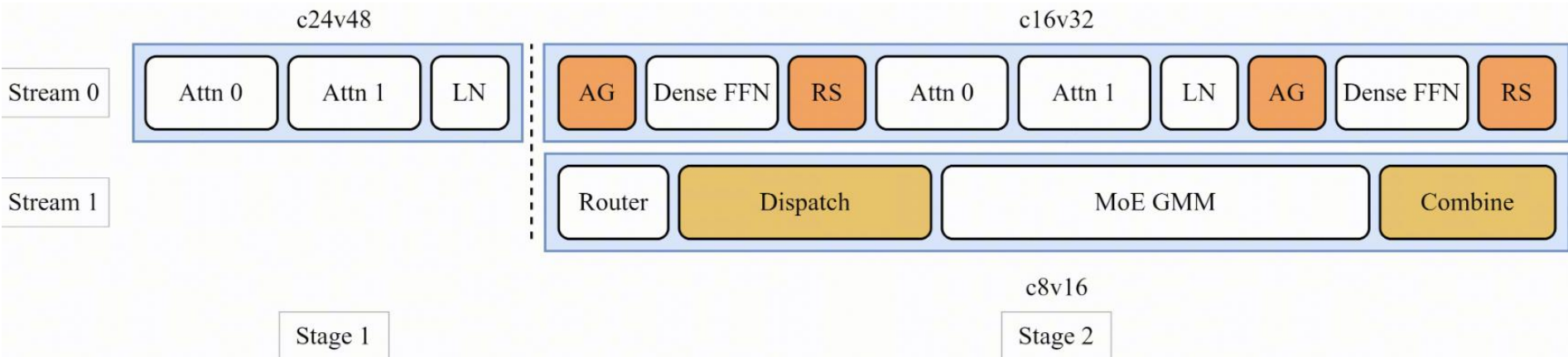
CANN

多流并行与控核

➤ Longcat模型支持四阶段并行策略，模型本身提供了并行空间



➤ Longcat模型调整后的多流并行和控核方案



- 多流并行: Decode低时延场景, CANN支持划分多个stream做并行计算, 多个stream上的计算形成overlap
- 控核: 多流并行时会出现所有核都被一个流占用的情况, 导致算子执行并行度降低, 因此需要把核分给不同的流使用

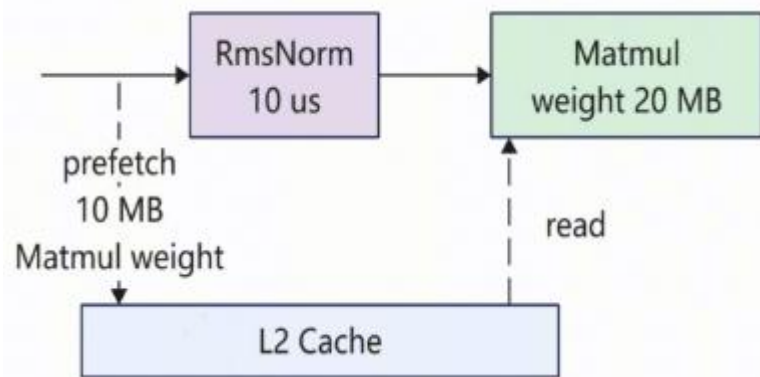
```
attn
layernorm
with npu_stream_switch(True, "1"):
    with limit_core_num(True, "8", "16"):
        router
        dispatch
        gmm
        combine
with limit_core_num(True, "16", "32"):
    dense
    attn
    layernorm
    dense
```


权重预取

- 问题：A2/A3 GM访存带宽1.6TB，Decode阶段存在明显的访存Bound问题

- 访存路径：AICore -> L2Cache -> GM
- L2Cache BW >> GM BW

- 在算子计算的同时，利用空闲的带宽，提前将一些访存bound算子的权重从GM搬运到L2 Cache中，提升算子性能

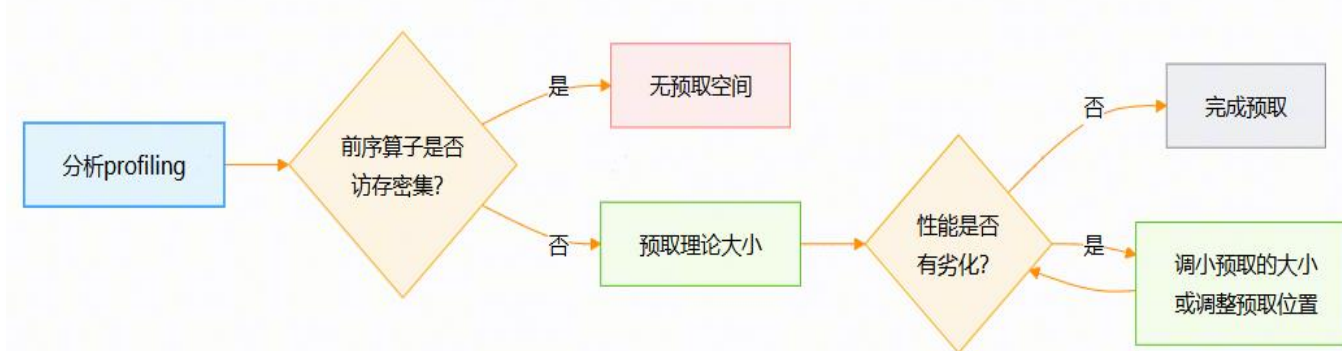


- 判断预取的大小和位置是否合理，可以通过profiling分析，如果与预取并行的算子性能有明显劣化，则需要进一步调整预取的位置，如没有可调整的位置，则需要减少预取的大小使得并行算子尽量不劣化。

```
s_cmo = torch.npu.Stream()
x = torch.randn(10000, 10000, dtype=torch.float32).npu()
y = torch.randn(10000, 1, dtype=torch.float32).npu()
add = torch.add(x, 1)

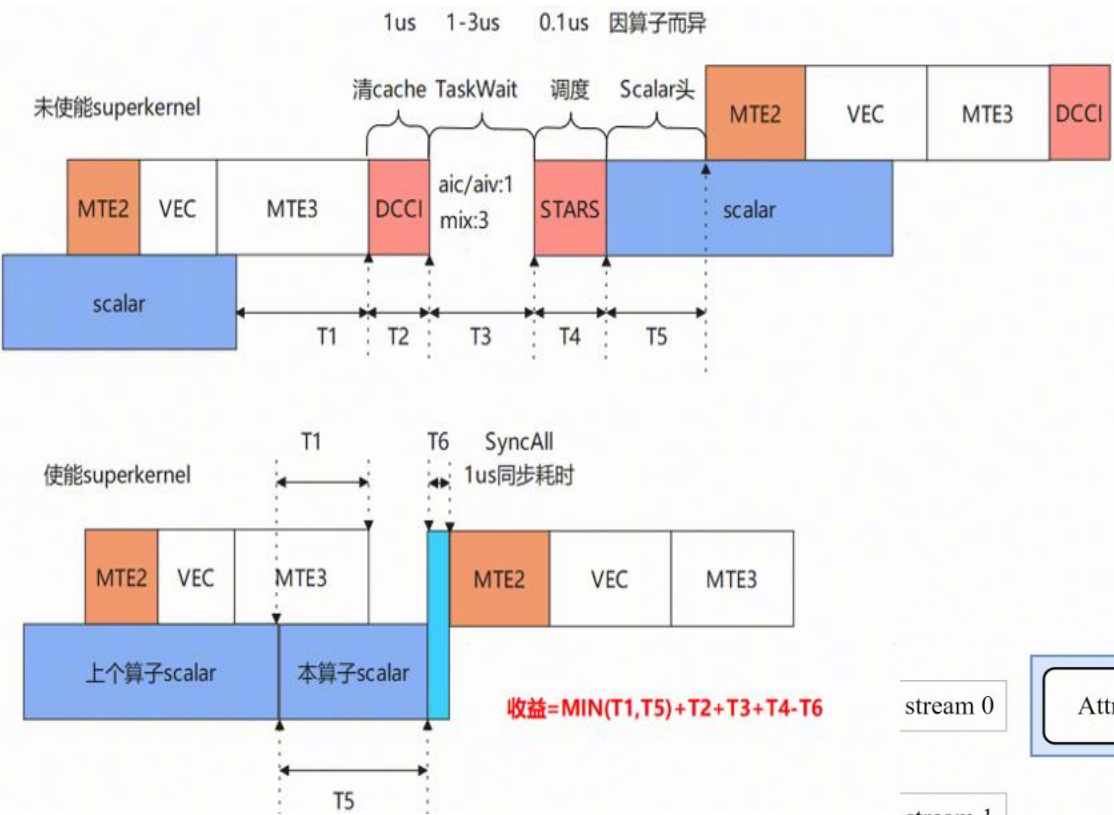
with torch.npu.stream(s_cmo):
    torch_npu.npu_prefetch(y, None, 10000000)
```

预取分析和调试的通用流程如下：



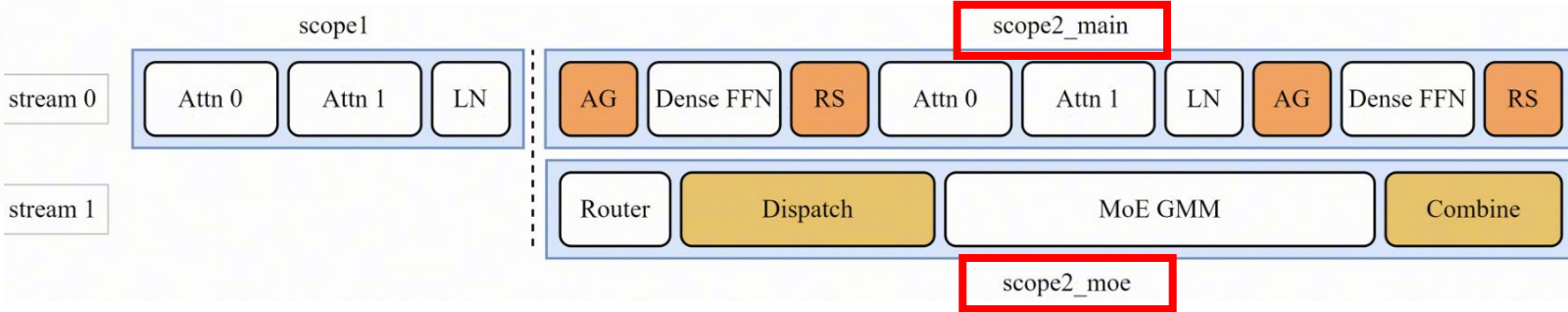
SuperKernel: 降低算子头开销

- A2/A3上算子调度头开销长，例如融合算子经验数据为5us
- 算子头开销，成为Decode低时延场景的关键瓶颈



- SuperKernel是一种算子二进制融合技术，与源码融合不同，它聚焦于内核函数（Kernel）的二进制调度方案优化，在已编译的二进制代码基础上融合创建一个超级Kernel函数（简称SuperKernel），以调用子函数的方式调用多个其它内核函数，达到优化计算任务、提升性能和资源利用率的目的。
- SuperKernel技术可以优化任务调度的等待时间和调度开销，还可以利用task间隙资源进一步优化算子头开销。

```
def forward(self, x1, x2, scale, offset, bias, pertoken_scale, weight_scale):  
    # 将 npu_quant_matmul和npu_dequant_swiglu_quant融合为superKernel, 标记为sp1  
    with torchair.scope.super_kernel("sp1", ""):  
        quant_matmul_res_origin = torch_npu.npu_quant_matmul(x1, x2, scale, offset  
        swiglu_res_origin = torch_npu.npu_dequant_swiglu_quant(quant_matmul_res_or  
        return swiglu_res_origin, quant_matmul_res_origin
```



目录

Part 1 模型切分部署策略

Part 2 极致推理优化技术

Part 3 Future Plan

下一步计划

- AFD分离技术，通过A2F/F2A算子进行节点间的数据交互，在2.0模型上端到端使能

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.



上CANN社区获取干货



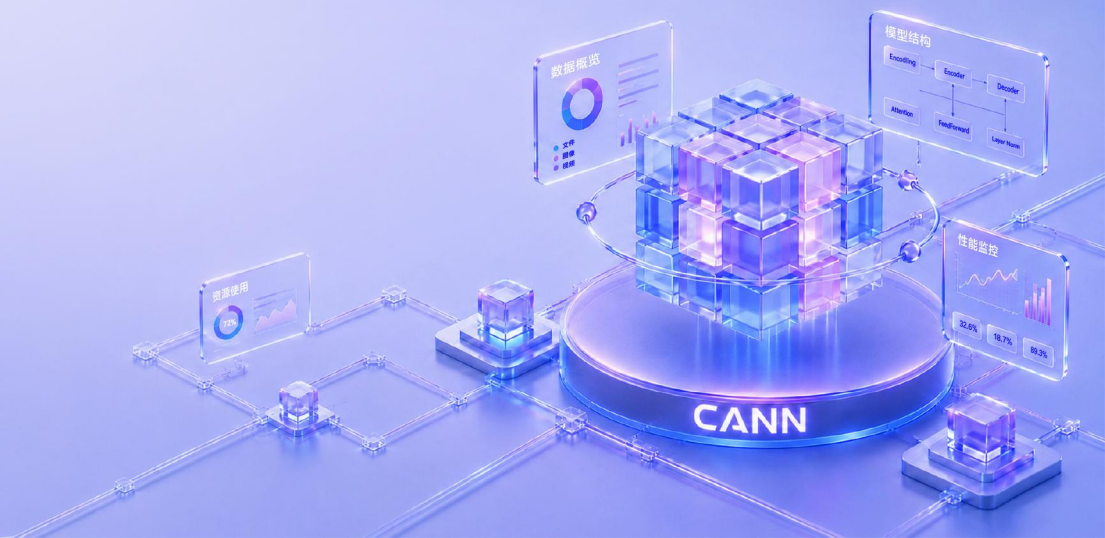
关注CANN公众号获取资讯





SanaVideo 模型: NPU 单卡部署与性能优化实践

李玮杰



目录

Part 1 背景与模型简介

Part 2 NPU适配与推理优化

Part 3 实机演练

什么是文生视频？

- 输入一段提示词，生成与描述内容匹配的视频

应用场景：

- AI视频创作与内容生产

痛点：

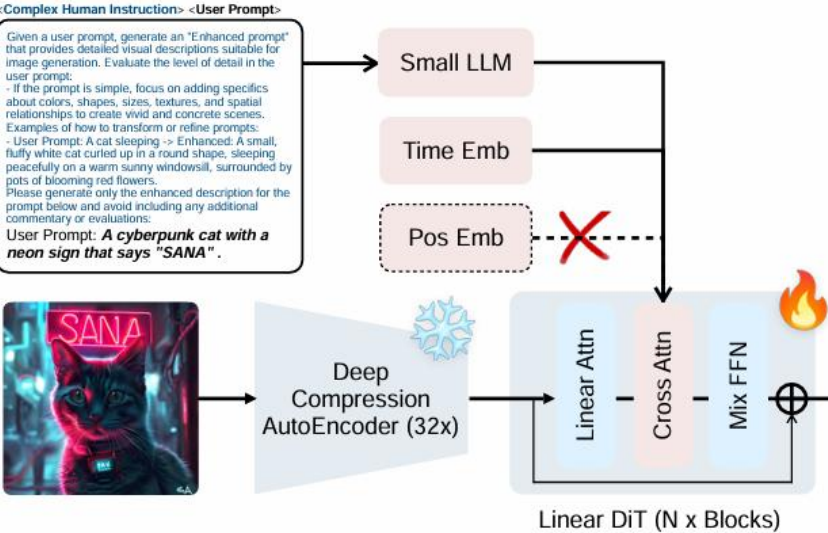
- 生成成本高
- 部署门槛较高，个人显卡显存不足



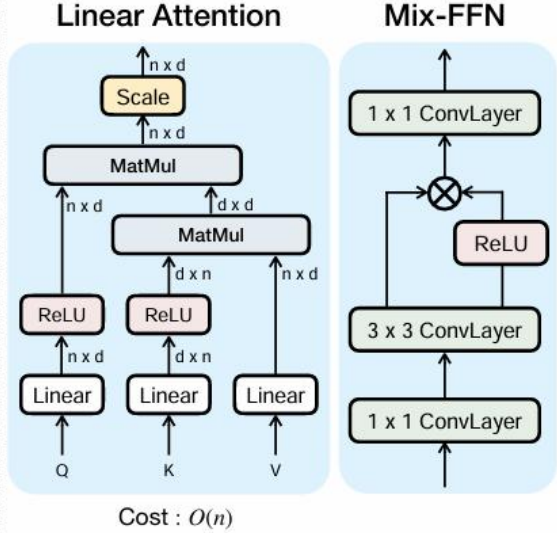
提供NPU平台和以sanavideo为例的demo方案：从 Prompt 到 MP4，文本编码、扩散采样与视频解码均在NPU单卡完成



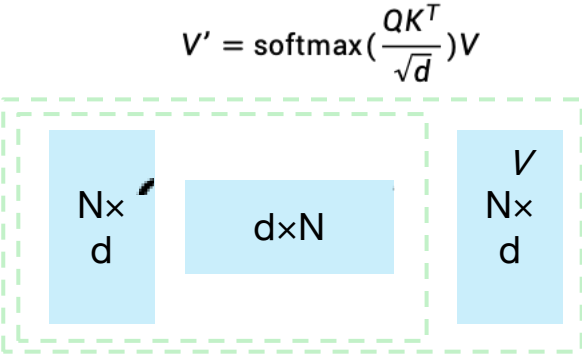
Linear DiT 将核心注意力复杂度降为线性



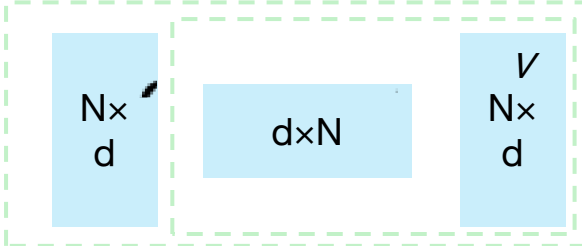
(a). Architecture overview of our Sana.



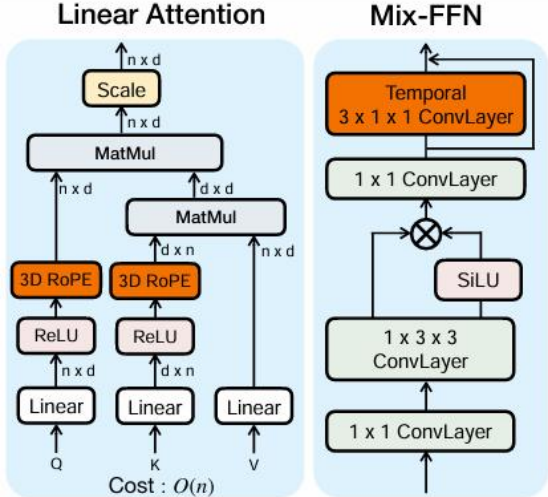
(b). Linear DiT Module.



$$O(N \cdot d \cdot N + N \cdot N \cdot d) = O(N^2 d)$$



$$O(N \cdot d \cdot d + d \cdot N \cdot d) = O(d^2 N)$$



(c). Linear DiT module

01 Linear Attention

以 ReLU 核重排 Q/K/V 计算，核心复杂度 O(N)

02 3D RoPE

在时间、高度、宽度三个维度注入位置信息

03 Temporal Conv

在 Mix-FFN 中加入时序卷积增强时序连续性

目录

Part 1 背景与模型简介

Part 2 NPU适配与推理优化

Part 3 实机演练

在npu环境跑通sanavideo模型推理：

01 环境准备

- CANN Toolkit（已提供）
- conda环境与torch_npu 以及其他依赖包
- 模型权重与离线路径准备

02 脚本准备

```
import torch_npu
from torch_npu.contrib import transfer_to_npu
```

将常见的CUDA API映射到NPU
将分布式后端从NCCL适配到HCCL

当前仓库的增量 patch 流程

保留原版 SanaVideo 主体流程，在模型构建前注入必要的 NPU 适配与优化。



先完成环境与脚本入口准备，后续融合算子替换和量化都在增量 patch 框架下展开

什么是融合算子？

针对 NPU 硬件特点，将多个原本独立执行的计算算子合并成一个新的算子，以减少调度开销、降低内存访问、提升推理性能。

融合前后对比

替换前

PyTorch实现，多步操作
每一步都会产生中间 tensor，增加读写和显存访问
更多 kernel 调度与中间结果读写

替换后

NPU 原生融合执行
固定计算模式由一个融合算子完成
减少算子数量、访存和调度开销

整网性能收益

npu_rms_norm	6%
npu_rotary_mul	1.5%
npu_fusion_attention	16%

```
class RMSNorm(torch.nn.Module):
    def _norm(self, x):
        return x * torch.rsqrt(x.pow(2).mean(self.norm_dim, keepdim=True) +
self.eps)
    def forward(self, x):
        ndim = x.dim()
        weight_shape = [1] * ndim
        weight_shape[self.norm_dim] = -1
        weight = self.weight.view(*weight_shape)
        return (weight * self._norm(x.float())).type_as(x)
```

```
def patch_rms_norm() -> None:
    norms = importlib.import_module("diffusion.model.norms")
    original_forward = norms.RMSNorm.forward
    def forward(self, x):
        if torch.npu.is_available() and torch_npu.__version__ >= "2.6.0":
            return torch_npu.npu_rms_norm(x, self.weight, epsilon=self.eps)[0]
        return original_forward(self, x)
    norms.RMSNorm.forward = forward
```

什么是量化？

用更低精度的数据表示模型参数和计算，用较小的精度损失换取更快的推理速度和更低的显存占用

910	INT8/INT4
950	INT8/FP8/MXFP8/MXFP4/HiF8

A4W4_MXFP4 配置采用按模块回退的混合策略，整网推理提速13.8%

模块精度

模块	A8W8_MXFP配置	A4W4_MXFP配置
Self Attention QKV	A8W8	A4W4
Cross Attention Linear	A8W8	A4W4
Attention 输出投影	A8W8	A8W8
1 × 1 Conv	A8W8	A8W8
计算输出	BF16	BF16



A4W4_MXFP4 配置



VBench精度验证

	subject consistency	motion smoothness	dynamic degree	aesthetic quality	imaging quality	overall consistency	avg
bf16	0.9672	0.9837	0.8000	0.6873	0.6930	0.2754	0.7344
A8W8	0.9663	0.9838	0.7583	0.6818	0.6932	0.2746	0.7263
A4W4	0.9646	0.9831	0.7667	0.6681	0.6874	0.2690	0.7232

来源: README.md 性能数据、patches/quant_module.py

目录

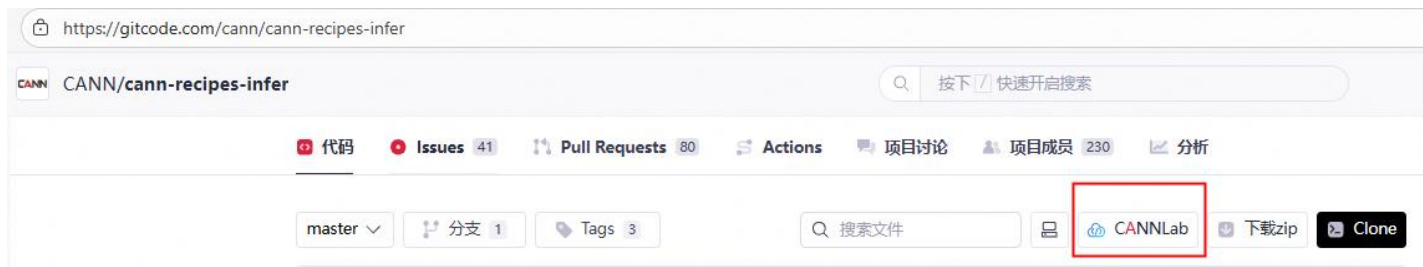
Part 1 背景与模型简介

Part 2 NPU适配与推理优化

Part 3 实机演练

基于一站式平台CANNLab，将环境、代码、权重和推理串成可复现闭环

<https://gitcode.com/cann/cann-recipes-infer>



Discussion



调查问卷



一键部署

bash infer_platform.sh

- 安装torch/torch_npu (conda环境中)
- 拉取SANA源代码
- 安装原始包依赖
- 下载模型权重
- 启动模型推理

→ output/sana_t2v_video_results/vis/*.mp4

常见问题

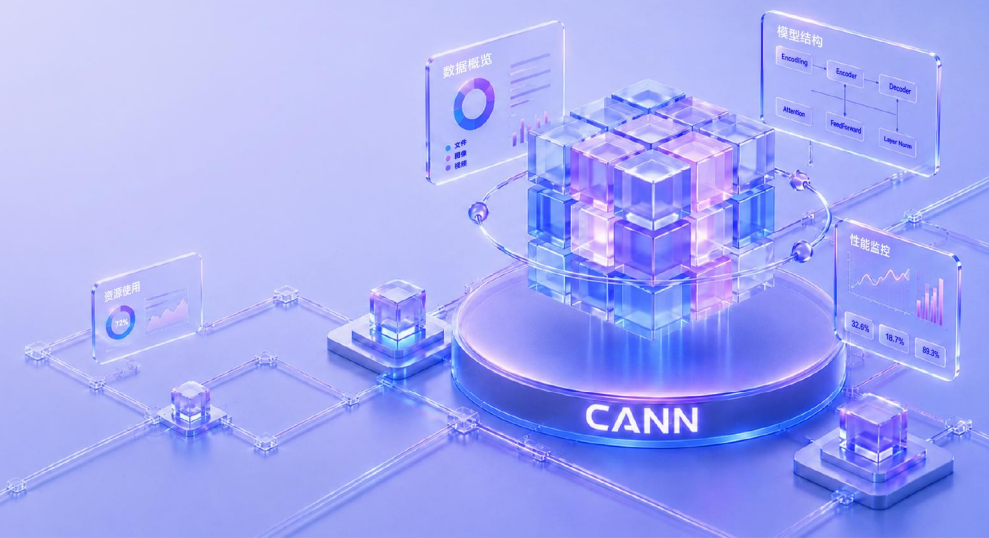
- 权重下载失败或超时?

推荐使用 aria2 + hfd 加速下载：sudo apt-get install -y aria2
增大下载超时时间：export HF_HUB_DOWNLOAD_TIMEOUT=600

- 卡时不够怎么办?

<https://gitcode.com/org/cann/discussions/114>

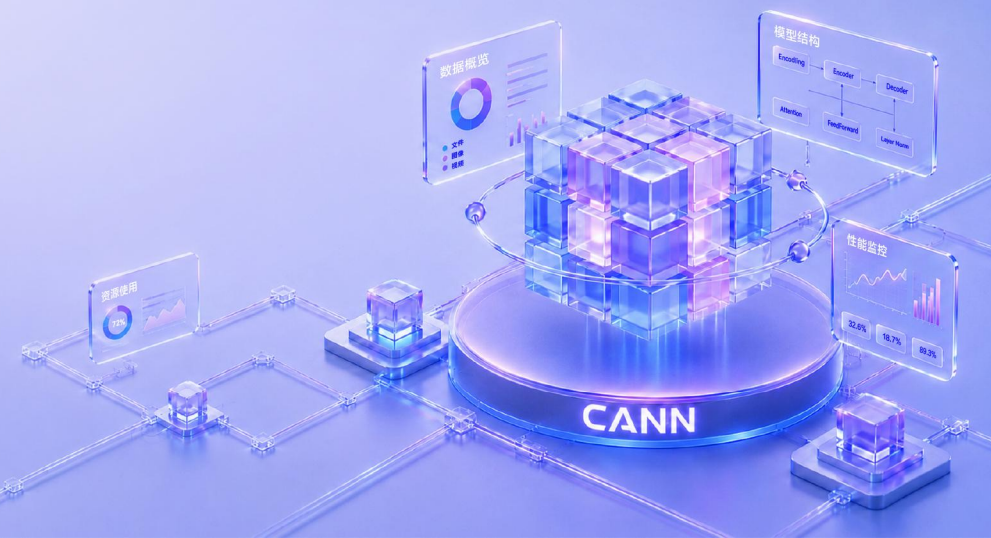
THANKS





DeepSeek-V4-Flash 单卡推理

袁源



CANN Tech Connect 2026

CANN开发者Meetup

DeepSeek-V4-Flash 单卡推理

Ascend NPU + Kunpeng CPU MoE Offload

单卡 64 GB GM 部署 43 层全 MoE 模型：热专家常驻 NPU，其余路由专家 offload 至 CPU，直接使用官方 MXFP4 权重

19 – 22.5

tok/s decode (A3 实测中位)

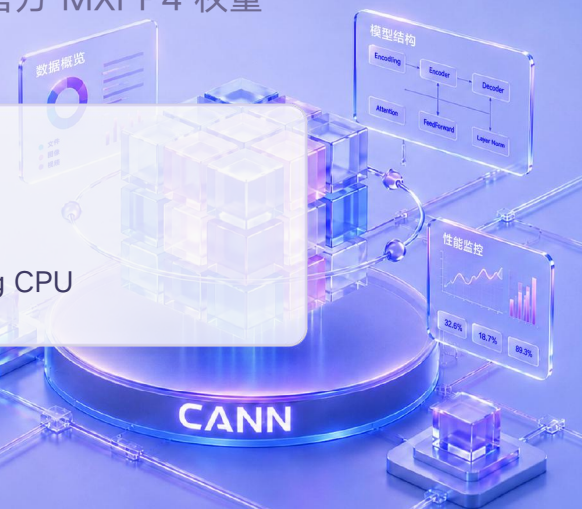
71.72%

GPQA-Diamond 三轮均值

单卡

A3 + 大内存 Kunpeng CPU

PR: gitee.com/cann/cann-recipes-infer/pull/682 • 分支: wenxuewuhd/cann-recipes-infer @ dsv4-npu-fullfeature-a3



43 层全 MoE 模型的单卡部署方案

DeepSeek-V4-Flash: first_k_dense_replace=0, 每层都是 MoE; 256 路由专家 / 层, 每 token 激活 6 个 + 1 个共享专家

约束条件

● 权重规模

43 层路由专家常驻内存约 137 GiB, 超出单卡 64 GB GM 容量

● 格式约束

官方权重为 MXFP4, NPU 仅支持 int8

● 带宽受限

bs=1 decode 是 GEMV: 每份权重只用一次, 算术强度约 3.8 OP/byte, 远低于 roofline 拐点 ~33

● 环境适配

NSA / 融合算子在不同 CANN 版本间调用约定不一致, 环境构建易出错

切分方案

NPU 侧

attention (MLA + NSA + Lightning Indexer)、router gate、共享专家、最热的 N=32 个路由专家、KV cache

CPU 侧

其余 224 个路由专家, kt-kernel LLAMAFILE 后端, 直接加载官方 MXFP4 转出的 GGUF

执行重叠

CPU MoE 走侧流, 通过 ACL host callback 接进 NPU graph, 与 NPU 计算并行

系统架构与单层数据流

单卡： A3（64 GB GM） + Kunpeng CPU（基准机 40 核 / 1 NUMA，也支持 K920 192 核 / 8 NUMA）



以上全部在 NPU 上执行

NPU 执行面（默认 N = 32 路由专家）

- fused_experts_npu (W8A8 int8 + fp32 per-channel scale)
- 共享专家与 router gate 常驻，不参与 offload
- KV cache 常驻 GM; attention backend = ascend
- depool: 不常驻整份 W8A8 池，AscendC kernel 现转 MXFP4→NZ

CPU 执行面（其余 224 路由专家）

- kt-kernel LLAMAFILE 后端，原生 MXFP4 GGUF（每层约 3.42 GiB）
- GEMV kernel: vqtbl1q_s8 查表 + vdotq_s32 (SDOT)
- 线程池按 NUMA 划分，KT_THREADPOOL_COUNT ≤ NUMA 节点数
- 激活在线量化至 Q8_0，为全链路唯一数值损失源

merge (topk_weights 加权合并) → linear + residual → 下一层
MoE 接进 NPU graph

桥: aclrtSubscribeReport + 轮询 aclrtProcessReport 的专用 poller 线程，把 CPU

两类 offload 方案与本方案的取舍

权重放置位置与计算位置的组合，决定瓶颈落在哪一类资源上

方案 A · compute-to-data

权重驻留 DDR，由 CPU 就地计算

计算位置	CPU
跨总线传输	仅回传激活，KB 量级
瓶颈资源	CPU 本地 DDR 带宽
延迟特性	无 stall，可与 NPU 计算重叠
算力上限	受限于 CPU 算力

方案 B · data-to-compute

miss 时将权重搬入 GM，由 NPU 计算

计算位置	NPU
跨总线传输	专家权重，MB 量级
瓶颈资源	CPU↔NPU 互联
延迟特性	miss 即 stall
算力上限	NPU 算力 + GM 带宽

本方案：A / B 混合

热专家采用 B：常驻 GM，跨 token 复用，搬运成本已摊销，且不产生 miss stall；冷专家采用 A：在 CPU 就地计算，不占用互联带宽。纯 B 方案仅在互联带宽极高或不存在互联的架构下成立，即 UMA（Mac / Strix Halo），这也是业界两条路线分野的物理原因。

纯 B 方案的可行性边界：单专家 MXFP4 13.37 MB × 每 token 264 个 slot ≈ 3.5 GB/token；达到 20 tok/s 需约 70 GB/s 的持续跨总线带宽，PCIe Gen4 ×16 理论上限约 32 GB/s，相差一个量级。



两份权重与量化基底一致性要求

CPU 侧使用 MXFP4 系官方发布格式，NPU 侧使用 int8 系硬件限制，二者均非出于性能目的的量化选择

NPU 侧 · W8A8 safetensors

- int8 权重 + fp32 per-channel scale
- 承载 attention / 共享专家 / router / 常驻热专家
- 启动时 MODEL_PATH 指向它
- 来源: modelscope · sgl-npu/DeepSeek-V4-Flash-W8A8

CPU 侧 · 官方原生 MXFP4 → GGUF

- E2M1 nibble + ue8m0 scale (K/32 分组)
- 转 GGUF 是 bit 级无损 repack, 不重新量化
- 每元素 0.53125 B; 单专家 13.4 MB; 43 层约 137 GiB
- 来源: huggingface · deepseek-ai/DeepSeek-V4-Flash



必须使用官方发布的 W8A8。

CPU 侧 GGUF 与 NPU 侧 W8A8 需为同一量化基底 (quarot 旋转)。自行用 modelslim 重新量化, 会导致输出异常且无任何报错。

转换约定 · nibble 排布

官方 ckpt 是 consecutive 排布 (byte i 存第 $2i / 2i+1$ 个 nibble), 上游 GGUF 是 half-block ($qs[j]$ 存第 $j / j+16$ 个) ——转换器需逐 32-group 重排; 约定不一致时不会报错, 仅表现为数值偏差, 以 `verify_mxfp4_layer.py` 的 bit-exact 对账为判据。

瓶颈定位：内存带宽受限，非算力受限

定量数据测自 A3 生产态：单卡 A3+ Kunpeng 40 核 / 1 NUMA, graph 与 side-stream 全开, 16k prompt, temp=0

3.8 OP/byte

decode(bs=1) 的算术强度

33 OP/byte

该机 roofline 拐点

142 GB/s

反推的实际 DRAM 带宽

155 GB/s

单 NUMA 带宽硬顶

带宽反推过程

每 token top-k slots ≈ 264 (6 $\times$ 43 层 + 共享)

resident 命中率 $H = 26.2\%$ (实测)

落到 CPU 的 slots $= 264 \times (1 - 0.262) \approx 195$

从 DRAM 读的权重 $= 195 \times 13.37 \text{ MB} \approx 2.61 \text{ GB}$

$2.61 \text{ GB} \div 18.3 \text{ ms} \approx 142 \text{ GB/s}$

结论：可行方向为减少 DRAM 读取字节数

- 142 / 155 GB/s: CPU MoE 已接近带宽饱和, 带宽利用率一侧基本无优化空间
- 优化路径只有两条: 降低单专家字节数 (MXFP4 半字节, 已实现); 减少落到 CPU 的专家数量 (提高命中率 H)
- CPU 侧算力存在余量, 制约因素为访存带宽

动态热专家常驻：命中率 H 的可控化

KT_DYNAMIC_RESIDENT=1，默认开启。按实际路由选取最热专家常驻 NPU，替代静态的前 32 个

静态 prefix-32

覆盖约 13% 的激活

实现简单、可回退，但专家 id 与热度无关

动态热专家常驻

覆盖量约为静态的 3×

按实际路由统计选取，需若干次请求暖机后收敛

当前实测

$H \approx 26 - 31\%$

长上下文启用 KT_HOT_TAIL_TOKENS=64
采样可达约 43%

对 decode 的作用机制

每命中一个专家即减少 13.37 MB 的 DRAM 读取。cpu_moe_wall 随 miss 比例近似线性变化： $\approx 24.8 \times (1 - H)$ ms。CPU 侧带宽已近饱和，H 为当前唯一可调量。

流式 prefill、depool 与 GGUF dedup

长序列 prefill 与 GM 占用的联合优化

长序列 prefill 流式加载

1

`KT_PREFILL_STREAM=1`

长序列 prefill 将全部 256 个专家流式搬入 NPU 计算：4096 prefill 约 14 s，hybrid 方式约 137 s，约 8×。触发阈值 `KT_PREFILL_STREAM_THRESHOLD=512`。

depool + AscendC MXFP4 算子

2

`KT_MXFP4_DEPOOL=1`

不再常驻整份 W8A8 池，改用 device 端 AscendC kernel（MXFP4 dequant + ND→NZ）现转，显著降低 GM 占用。代价：需现场编译 `libmxfp4fused.so`。

GGUF 去重

3

`KT_MXFP4_GGUF_DEDUP=1`

NPU 侧直接复用 CPU 已 mmap 的 GGUF，减少一份常驻内存。

残余开销：流式 prefill 存在约 15 s 固定成本（每次请求需将全套专家权重由 DDR 搬入 GM），基本不随 prompt 长度增长，是长 prompt TTFT 的主要构成，也是后续可优化项。

CPU↔NPU 执行重叠与基础支撑项

side-stream 净收益: 9.4 ms / token

- 约 4.8 ms: 被 NPU resident GEMM 掩盖的 CPU 计算
- 约 4.6 ms: 避免了串行模式下 host 回调挂主流、逐层阻塞启动流水的开销 (来源为消除 host 阻塞, 非计算掩盖)
- attention / NSA 在 fork 之前完成, 存在数据依赖, 结构上无法进入侧流

开关: KT_SIDE_STREAM /
KT_SHARED_EXPERTS_STREAM (默认 1)

- CPU MoE 的 host callback 走侧流, 与 NPU 上的常驻专家 / 共享专家计算重叠
- 闭合方式: ascend_callback_worker 起后台线程做 aclrtSubscribeReport + 循环 aclrtProcessReport
- eager 模式下异步提交未 flush 会导致输出异常, 排障时设 KT_FORCE_SYNC_SUBMIT=1

权重加载加速

zero-copy mmap + 并行重排, 43 层约 47 s

triton × ascend 自动回退

版本错配时自动探测并回退到数值等价的纯 PyTorch 实现, 无需任何开关

NSA compressor 双版本兼容

CANN 9.0.0 公开 18 参 single-state / 8.5.0 私有 19 参 split-state, 由拉起脚本按已装 CANN 自动派生

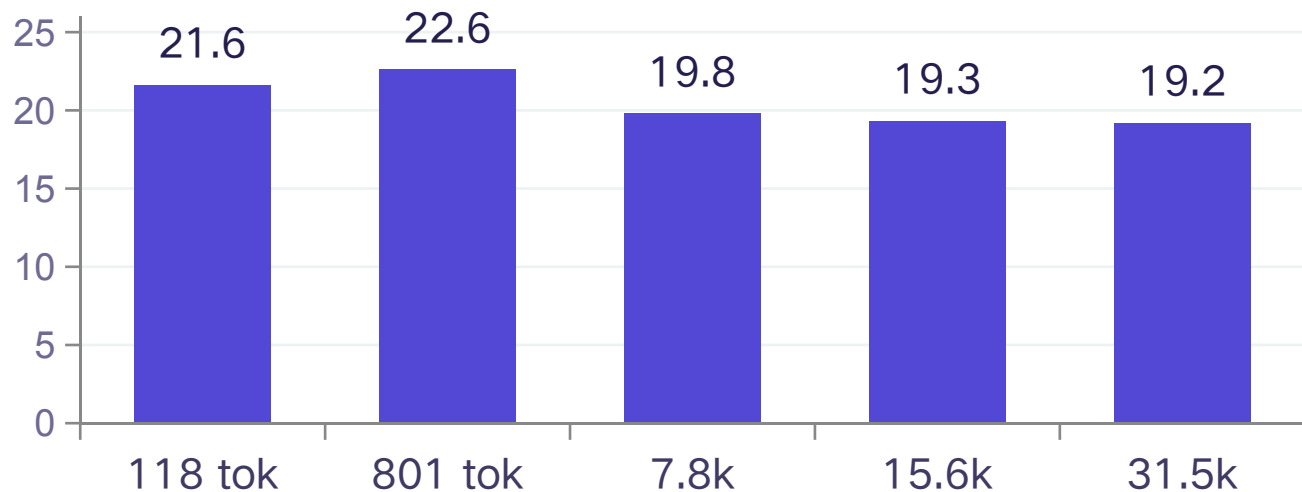
综合效果: cpu_moe_wall 从早期的 22 - 27 ms 降到约 18.3 ms (A3 生产态, H ≈ 26%)

当前状态

实测性能：A3，graph 模式，bs=1

配置为默认配置：KT_CPUINFER=32、KT_THREADPOOL_COUNT=1、KT_NUM_GPU_EXPERTS=32，depool + 流式 prefill + 动态常驻全开

稳态 decode (inter-token 中位, tok/s)



19 – 22.5 tok/s

decode 中位区间；单 token 快端 23 – 25

18.3 ms

cpu_moe_wall / token (早期为 22 – 27)

15.4 – 20.3 s

prefill 130 / 1k / 8k / 16k / 32k (暖机后)

吞吐下降的常见原因（按出现频率排序）

共享机邻居进程争抢 DRAM 带宽（最常见） · 未暖机（动态常驻需若干请求收敛） · NPU 卡被其它容器占用 · 线程池与 NUMA 拓扑不匹配 · 并发请求（--max-running-requests > 1 触发 NPU 资源争抢） · 上下文增长 · 冷盘首次启动

CANN

长上下文可通过 KT_HOT_TAIL_TOKENS=64 获得 +3%~8%（短 prompt 有损失，默认关闭） · 流式 prefill 约 15 s 固定开销，几乎不随长度增长

业界方案横向对比（单卡 + offload 路线）

外部数字为公开二手源（新闻 / 博客 / 官方 spec，2026-06 调研），单用户口径、多为区间，仅作量级参照

方案	硬件	模型 / 量化	单用户 decode	瓶颈
本方案	Ascend 910C(A3) 64 GB GM + Kunpeng CPU	DSV4-Flash / MXFP4 同源 4-bit	19 – 22.5 tok/s	CPU 本地 DDR
ktransformers	RTX 4090D 24 GB + Xeon + 1 TB DDR5	DSV3 / R1 671B / 4-bit GGUF	~14 tok/s (dsv3)	CPU DDR + 同步开销
llama.cpp (-ot 专家外放)	单 RTX 4090 24 GB + DDR	DSV3 671B / Q4	~12 tok/s	CPU 本地 DDR
Apple M3 Ultra (UMA)	512 GB 统一内存, 819 GB/s	R1 671B / 4-bit	~16 – 20 tok/s	UMA 带宽
AMD Strix Halo (UMA)	128 GB 统一内存 (标称 ~256 GB/s)	同款 DSV4-Flash / IQ1_S 1.6-bit	~1 – 2 tok/s	UMA 容量/带宽

差异化定位

在「独立 GM + 大内存 CPU」架构下运行同款 DSV4-Flash，并保持官方 4-bit 同源量化、不降低精度基底；UMA 路线受 128 GB 容量限制需压缩至 1.6-bit，吞吐降至 1 – 2 tok/s。

同类方向的外部印证

vLLM 的 MoE offload RFC (#38256) 用「GPU 侧热专家 cache + LFRU 驱逐 + 跨层预测」，与本方案的热专家常驻池 + 预测预取基本同构；学术侧 PreScope、SpecOffload 也是同一方向。

外部独立交叉验证：AMD 的公开文章给出 DSV4-Flash = 284B 总参 / 13B 激活，与我方按 config 推算的规模互相印证



命中率 H 与 decode 上限：约 +37% 的余量

模型：cpu_moe_wall(H) $\approx$ 24.8 $\times$ (1 - H) ms，其中约 4.8 ms 能被 NPU 的 resident GEMM 窗口掩盖；NPU 临界路径约 36 ms（16k 口径）

resident 命中率 H	cpu_moe_wall	露出在关键路径上的 CPU	TPOT	decode
26%（当前）	18.3 ms	13.5 ms	49.9 ms	20.1 tok/s
50%	12.4 ms	7.6 ms	44.0 ms	22.7 tok/s (+13%)
60%	9.9 ms	5.1 ms	41.5 ms	24.1 tok/s (+20%)
$\approx$ 80%（上限）	4.8 ms	0（被完全掩盖）	36.4 ms	$\sim$ 27.5 tok/s (+37%)

H $\approx$ 80% 之后的瓶颈转移

CPU MoE 被 NPU 完全掩盖，decode 达到 NPU 临界路径上限；继续提高命中率对 decode 无收益，瓶颈转为 attention / NSA / resident GEMM 本身，属本项目当前范围之外。

口径说明

H 提高会使更多 slot 落入 resident GEMM，将 36 ms 临界路径抬高数 ms，实际上限更可能为 $\sim$ 25 - 27 tok/s；短上下文 NPU 临界路径更小、天花板更高，长上下文更低。以上均为 16k 口径。

后续优化方向：提高命中率、降低搬运字节

CPU MoE 带宽已近饱和，两类可行方向：减少落到 CPU 的专家数（提高 H）与降低单专家字节数；H 的换算依据见上一页 TPOT(H) 模型

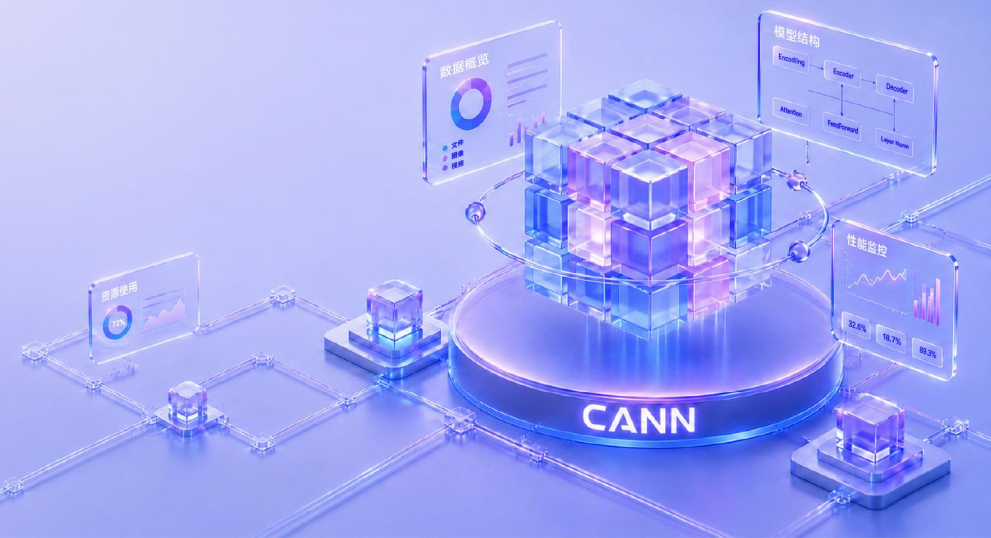
1	热专家常驻策略优化 CPU 侧主要方向	跨请求频率统计、按层差异化、hot-tail 采样。当前动态常驻的 H 为 26 – 31%，提高至 50 – 60% 是 decode 的主要收益来源。	H → 50 – 60% +13~20%
2	专家预测 + H2D 预取 在研 / 论证中	提前一个 token / 一层预测将命中的 expert，利用当前计算时间异步完成 H2D 预取，使搬运被计算掩盖。	降低暴露于 TPOT 的搬运
3	加大 KT_NUM_GPU_EXPERTS 直接，代价为 GM 占用	常驻数由 32 增至更多（32→48 约增加十余 GB）。达到 80% 上限主要依赖该路径，受静态权重 48.3 GB / 余量约 8.8 GB 约束。	H → 80% 需与 KV 权衡
4	CPU 侧更低比特 MoE 带宽侧的另一条路径	cpu_moe_wall 与每元素字节数近似成正比，MXFP4 当前为 0.53125 B/元素；进一步降至 3-bit / 2-bit 类格式可近似线性压缩搬运量，与提高 H 相互独立、可叠加。 约束：kernel 仅使用 ARMv8.2 dotprod（查表 + SDOT），新格式需配套 GEMV kernel 并保持 llamafire 泛化路径；精度须重做 bit-exact 与端到端对账。	字节数 ↓ 近似线性收益

其它在研 / 计划：MTP（模型自带 num_nextn_predict_layers=1，当前禁用） · 推广到 dspark / dflash 等同类模型 · 可由社区参与：热池策略、预取算法、更低比特 kernel、prefill 固定开销压缩、机型适配



THANKS

DeepSeek-V4-Flash 单卡推理 · Ascend NPU + Kunpeng CPU MoE Offload



CANN Tech Connect 2026

CANN开发者Meetup

2026.8.8 中国·杭州

主办单位：CANN开源社区

协办单位： AtomGit

